

Министерство образования и науки Российской Федерации
Сибирский федеральный университет

**ВЫСОКОПРОИЗВОДИТЕЛЬНЫЕ ВЫЧИСЛЕНИЯ
НА ГРАФИЧЕСКИХ ПРОЦЕССАХ.
ЛАБОРАТОРНЫЙ ПРАКТИКУМ**

Учебно-методическое пособие

Электронное издание

Красноярск
СФУ
2017

УДК 004.383(07)
ББК 32.971.322я73
В932

Составитель: **Удалова Юлия Васильевна**

В932 Высокопроизводительные вычисления на графических процессах. Лабораторный практикум : учеб.-метод. пособие / сост. : Ю.В. Удалова. – Электрон. дан. – Красноярск : Сиб. федер. ун-т, 2017. – Систем. требования: PC не ниже класса Pentium I ; 128 Mb RAM ; Windows 98/XP/7 ; Adobe Reader V8.0 и выше. – Загл. с экрана.

Содержит курс лабораторных работ по дисциплине «Высокопроизводительные вычисления на графических процессорах» с элементами теории и примерами программ, основанными на применении библиотеки OpenCL.

Предназначено для направления 09.04.01.01 «Высокопроизводительные вычислительные системы».

УДК 004.383(07)
ББК 32.971.322я73

© Сибирский федеральный
университет, 2017

Электронное учебное издание

Подготовлено к публикации издательством
Библиотечно-издательского комплекса

Подписано в свет 28.06.2017. Заказ № 1605
Тиражируется на машиночитаемых носителях

Библиотечно-издательский комплекс
Сибирского федерального университета
660041, г. Красноярск, пр. Свободный, 82а
Тел. (391)206-26-67; <http://rio.sfu-kras.ru>
E-mail: publishing_house@sfu-kras.ru

Оглавление

Введение.....	4
Лабораторная работа №1. Введение в OpenCL	6
Лабораторная работа №2. Работа с ядром и памятью в OpenCL.....	30
Лабораторная работа №3. Синхронизация и защита критических сек- ций.....	43
Лабораторная работа №4. Многомерные массивы. Изображения.....	48
Лабораторная работа №5. События. Динамическое составление OpenCL программы	71
Лабораторная работа №6. Распараллеливание сложных алгоритмов.....	80

Введение

Организация высокопроизводительных вычислений требует применения параллельного программирования и соответствующих библиотек совместно с использованием высокопроизводительных аппаратных ресурсов для выполнения параллельных программ.

Современный персональный компьютер предоставляет многоядерный процессор и таким образом может осуществлять параллельное выполнение как нескольких задач (приложений), так и одной задачи (приложения), разбитой разработчиком на несколько параллельно выполняющихся параллельных примитивов (потокaв/нитей, процессов).

Существуют подходы к организации высокопроизводительных вычислений, выполняющие параллельную программу на суперкомпьютере или на кластере (совокупности персональных компьютеров, объединенных сетью).

Существуют и альтернативные подходы, позволяющие задействовать для высокопроизводительных вычислений единственный персональный компьютер и при этом превысить производительность центрального процессора, тем более, что такая производительность упирается в физические ограничения. Суть этих альтернативных подходов состоит в выполнении параллельных программ на графической карте (графическом процессоре, видеокарте, видеочипе), которой обладает каждый современный персональный компьютер. Также возможны высокопроизводительные вычисления на персональном компьютере, объединяющем несколько видеокарт, а также сервере или суперкомпьютере, вычислительные мощности которых опираются на применении нескольких (возможно, десятков) видеокарт.

Как было замечено выше, максимальная частота центрального процессора (CPU - central processing unit) упирается в физические ограничения. При этом в персональном компьютере дополнительно используется и графический процессор (GPU - graphics processing unit), который даже в бюджетной модели содержит: мультипроцессор с четырьмя-десятью ядрами, десятки дополнительных сопроцессоров, а также персональную память. Хотя частота мультипроцессора и (особенно) сопроцессоров GPU чаще всего уступает частоте CPU, видеочип способен обрабатывать отдельные виды параллельных задач значительно быстрее центрального процессора, благодаря намного большему числу

вычислительных ядер в видеочипе. Объем памяти современной видеокарты сравним с объемом оперативной памяти персонального компьютера для офисных задач, при этом на видеокарте применяется более быстрая память, т.е. обеспечивается большая пропускная способность операций с данными во время выполнения задачи.

В отдельных источниках можно прочесть о вычислительных задачах, которые при распараллеливании на CPU выполняются за несколько дней (или месяцев), тогда как при распараллеливании на GPU эти же задачи решаются за несколько минут. Видеокарта изначально предназначена для получения исходных данных текстур и полигонов, их параллельной обработки и выдачи выходных данных. Однако этот же ресурс можно использовать для любых других вычислений, не связанных в обязательном порядке с компьютерной графикой. Например, высокоэффективно будут распараллеливаться на видеокарте задачи обработки больших массивов данных (особенно если это однотипная обработка над каждым элементом, не требующая при этом длительного ожидания данных из других элементов). Однако заметим, что существуют и такие задачи, распараллеливание которых на GPU не даст существенного прироста эффективности в сравнении с их распараллеливанием на CPU.

Итого, приведенные аппаратные особенности делают графические процессоры (GPU) высокоэффективными для быстрого решения определенных классов задач.

Основные современные программные библиотеки для распараллеливания задач общего назначения на GPU это:

- более простая в изучении и использовании разработчиком OpenACC (Open Accelerators), не обладает свободной лицензией (хотя может быть получена пробная 90-дневная версия или годовая студенческая лицензия, предполагающая работу в ОС Linux);
- CUDA (Compute Unified Device Architecture), распространяется свободно, предназначена для параллельных вычислений на видеокартах NVIDIA восьмого поколения и выше;
- OpenCL (Open Computing Language - открытый язык вычислений), распространяется свободно, позволяет проводить параллельные вычисления на центральном или графическом процессоре, при этом возможно использование не только видеокарт NVIDIA, но и AMD.

Данное пособие описывает применение библиотеки OpenCL для организации высокопроизводительных вычислений на GPU.

Лабораторная работа №1

Тема: Введение в OpenCL.

Цели: Получить навык инсталляции библиотеки OpenCL. Изучить базовые термины и программные конструкции OpenCL.

Базовая информация об OpenCL

OpenCL (Open Computing Language - открытый язык вычислений) - библиотека, расширяющая функционал языков C, C++, C# (и одновременно накладывающая на них ряд ограничений), служащая для организации параллельных вычислений на объединенных мощностях центрального и графических процессоров (т.е. поддерживающая организацию гетерогенных вычислений).

С помощью OpenCL возможны параллельные вычисления на высокопроизводительных видеокартах компаний NVIDIA и AMD. Возможно ли задействование для параллельных вычислений именно вашей видеокарты, можно будет узнать с помощью специальных программных вызовов. При наличии проблем, ряд источников рекомендует установку новых драйверов для используемой видеокарты.

Электронные ресурсы OpenCL

- 1) Документация OpenCL. <http://www.khronos.org/opencv/>
- 2) Информация об OpenCL, инсталляция OpenCL (сайт AMD). <http://developer.amd.com/tools-and-sdks/opencv-zone/>
- 3) Информация об OpenCL, инсталляция OpenCL (сайт NVIDIA). <https://developer.nvidia.com/opencv>

Пример инсталляции OpenCL

Здесь в качестве источника для инсталляции выбран сайт NVIDIA, установка производится под ОС Windows для среды разработки Microsoft Visual Studio.

На сайте NVIDIA скачиваем любой пример (рис. 1), он будет содержать и необходимые заголовочные файлы (.h) и файлы библиотек (.lib) для C/C++. (Поддержка OpenCL для C# дело рук энтузиастов.)

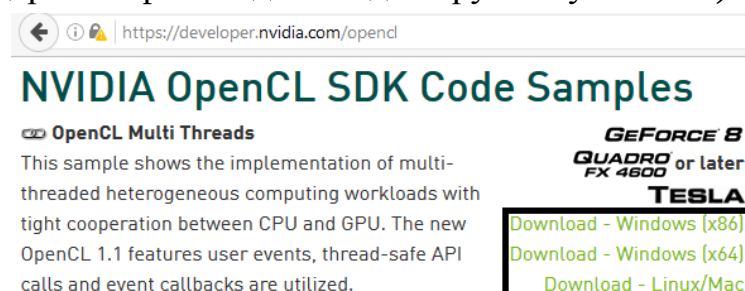


Рис. 1. Установка OpenCL. Источник сайт NVIDIA

Распаковываем скаченный архив и помещаем папку в любую выбранную нами директорию. Рассмотрев содержимое скаченной папки можно обнаружить:

- и запустить исполняемый .exe файл (это означает только запуск скаченного примера программы на OpenCL, а не установку библиотеки);
- файл проекта для Visual Studio .vcproj и открыть проект примера в вашей версии среды Visual Studio, если она безошибочно проводит компиляцию и запуск примера, то его можно использовать как каркас для написания вашего кода, не выполняя нижеописанные операции;
- заголовочные файлы (.h) и файлы библиотек (.lib), если сборка проекта, не прошла успешно, то нужно запомнить пути к папкам, где расположены эти файлы.

В примере заголовочные файлы расположены в "C:\Users\Ulk\Downloads\oclMultiThreads\NVIDIA GPU Computing SDK\OpenCL\common\inc" (здесь имеется файл oclUtils.h и папка CL с набором остальных заголовочных файлов, среди которых cl.h). Файл библиотеки (OpenCL.lib) расположен в "C:\Users\Ulk\Downloads\oclMultiThreads\NVIDIA GPU Computing SDK\OpenCL\common\lib\Win32". Могут быть и другие папки с файлом OpenCL.lib, например не "Win32", а "x64", "x86".

В среде Visual Studio создадим новый проект (тип проекта - консольное приложение Win32), в главном меню активируем пункт Проект→Свойства.

1) В раздел C/C++→Общие→"Дополнительные каталоги включаемых файлов" добавим путь к заголовочным файлам "C:\Users\Ulk\Downloads\oclMultiThreads\NVIDIA GPU Computing SDK\OpenCL\common\inc" (рис. 2).

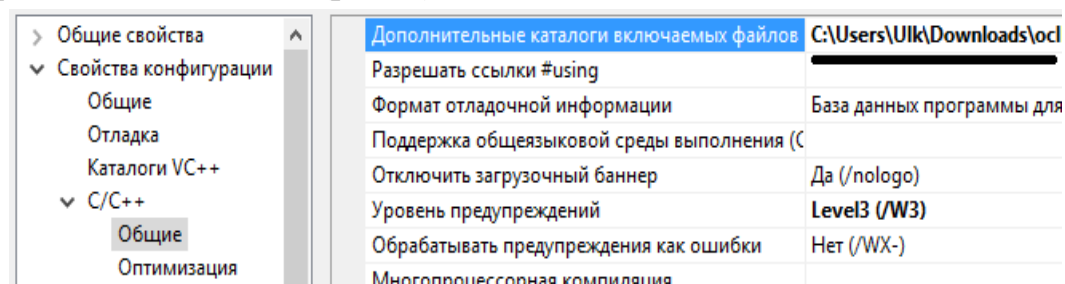


Рис. 2. Настройка пути к заголовочным файлам

2) В раздел Компоновщик→Общие→"Дополнительные каталоги библиотек" добавим путь к файлу библиотеки "C:\Users\Ulk\Downloads\oclMultiThreads\NVIDIA GPU Computing

SDK\OpenCL\common\lib\Win32" (рис. 3).

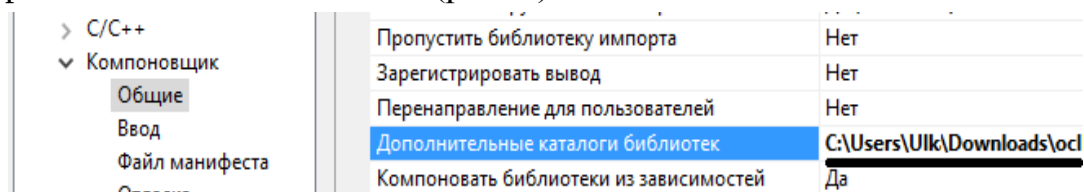


Рис. 3. Настройка пути к файлам библиотек

3) В раздел Компоновщик→Ввод→"Дополнительные зависимости" добавим (не стирая имена других библиотек) " OpenCL.lib;" (рис. 4).

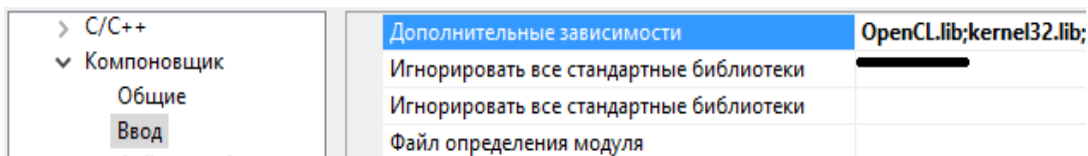


Рис. 4. Настройка файлов библиотек

Применяем внесенные в свойства проекта изменения, добавляем в код проекта подключение библиотеки `#include "CL/cl.h"`, если компиляция не приводит к ошибкам (например, не найден `cl.h`, не найден `cl_platform.h` и т.п.), значит все пути прописали корректно.

Для дополнительной проверки можно выполнить следующий код.

```
cl_int err; cl_uint num;
err = clGetPlatformIDs(0, 0, &num);
if (err == CL_SUCCESS && num!=0)
    printf("\nDetected OpenCL platforms.\n");
else printf("\nError.\n");
```

Этапы разработки OpenCL программы

1. Определение платформы

Платформа определяет конкретную реализацию программного интерфейса OpenCL. OpenCL платформ на одной ЭВМ может быть несколько, например, из-за наличия нескольких видеокарт, программных пакетов OpenCL (в том числе и от разных разработчиков), версий OpenCL.

Функции для работы с платформами

cl_int clGetPlatformIDs(cl_uint num_entries, cl_platform_id *platforms, cl_uint *num_platforms)

Функция предоставляет информацию о доступных платформах и возвращает CL_SUCCESS если вызов прошел успешно.

num_entries - количество записей, которые нужно добавить в массив платформ platforms, если platforms!=NULL, а если массив платформ не запрашивается (platforms=NULL), то num_entries приравнивается нулю.

platforms - указатель на массив платформ cl_platform_id, если platforms=NULL, массив платформ не заполняется.

num_platforms - по этому указателю записывается общее количество доступных платформ, если указать NULL опция не будет задействована.

При получении списка доступных платформ, информацию о каждой отдельной платформе можно узнать с помощью следующей функции.

cl_int clGetPlatformInfo(cl_platform_id platform, cl_platform_info param_name, size_t param_value_size, void *param_value, size_t *param_value_size_ret)

platform - платформа, о которой запрашивается информация.

param_name - константа, определяющая какую именно информацию о платформе запрашивает разработчик, это может быть:

CL_PLATFORM_PROFILE (тип char [])

CL_PLATFORM_VERSION (тип char [])

CL_PLATFORM_NAME (тип char [])

CL_PLATFORM_VENDOR (тип char [])

CL_PLATFORM_EXTENSIONS (тип char [])

CL_PLATFORM_HOST_TIMER_RESOLUTION (тип cl_ulong, отсутствует в ранних версиях OpenCL).

param_value - указатель на void (это означает, что в param_value можно указывать адреса значений произвольных типов, а обратите внимание, что типы запрашиваемых значений по param_name не одинаковы), по которому функция запишет ответ на запрос, указанный в параметре param_name.

param_value_size - размер в байтах значения, указанного в param_value.

param_value_size_ret - по этому указателю записывается реальный размер в байтах значения, записанного в param_value.

Функция возвращает CL_SUCCESS если вызов прошел успешно. Если в качестве аргументов-указателей назначать NULL (0), такая опция функции не будет задействована. Указанная пара свойств относится к большинству OpenCL функций.

Запустим нижеописанную программу и узнаем количество платформ и их параметры на используемой ЭВМ.

```
#include "locale.h"
#include "CL/cl.h"
int _tmain(int argc, _TCHAR* argv[])
```

```

    {   setlocale(LC_ALL, "rus") ;
        cl_int err; cl_uint num;
        err = clGetPlatformIDs(0, 0, &num); //запрос числа OpenCL
платформ
        if ( err == CL_SUCCESS && num!=0 )
            printf("\nОпределено %d OpenCL платформ.\n", num);
        else
            { printf("\nОшибка вызова clGetPlatformIDs или отсутствуют
OpenCL платформы.\n"); return -1 ; }

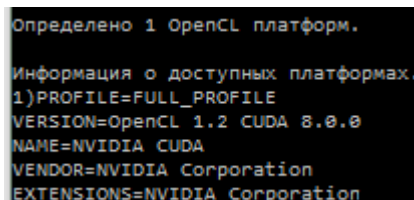
        //массив на num платформ
        cl_platform_id *platforms = new cl_platform_id [num] ;
        //запрос массива OpenCL платформ
        err = clGetPlatformIDs(num, platforms, 0);
        if ( err != CL_SUCCESS )
            { printf("\nОшибка вызова clGetPlatformIDs.\n"); return -1 ; }

        printf("\nИнформация о доступных платформах." ) ;
        char str[250] ;
        for (int i=0 ; i<num ; i++) //цикл обхода всех платформ
        {   printf("\n%d",i+1) ;
            clGetPlatformInfo(platforms[i], CL_PLATFORM_PROFILE,
250, (void *)str, 0);
            printf("PROFILE=%s",str) ;
            clGetPlatformInfo(platforms[i],
CL_PLATFORM_VERSION, 250, (void *)str, 0);
            printf("\nVERSION=%s",str) ;
            clGetPlatformInfo(platforms[i],    CL_PLATFORM_NAME,
250, (void *)str, 0);
            printf("\nNAME=%s",str) ;
            clGetPlatformInfo(platforms[i],
CL_PLATFORM_VENDOR, 250, (void *)str, 0);
            printf("\nVENDOR=%s",str) ;
            clGetPlatformInfo(platforms[i],
CL_PLATFORM_EXTENSIONS, 250, (void *)str, 0);
            printf("\nEXTENSIONS=%s",str) ;
        }
    }

```

```
delete [num] platforms ;  
return 0;  
}
```

Программа при запуске выводит список доступных OpenCL платформ и их параметры (рис. 5). Если найдено несколько платформ, для произведения дальнейших вычислений нужно будет выбрать одну из них в коде программы.



```
Определено 1 OpenCL платформ.  
Информация о доступных платформах.  
1)PROFILE=FULL_PROFILE  
VERSION=OpenCL 1.2 CUDA 8.0.0  
NAME=NVIDIA CUDA  
VENDOR=NVIDIA Corporation  
EXTENSIONS=NVIDIA Corporation
```

Рис. 5. Выполнение программы, выводящей список OpenCL платформ

Как видно из рис. 5, при запуске на конкретной ЭВМ была обнаружена только одна OpenCL платформа, значит для дальнейших вычислений можно выбрать только платформу `platforms[0]`.

2. Определение устройств

`cl_int clGetDeviceIDs (cl_platform_id platform, cl_device_type device_type, cl_uint num_entries, cl_device_id *devices, cl_uint *num_devices)`

Функция заполняет массив устройств `devices` и заносит по указателю `num_devices` количество элементов (устройств) в данном массиве. В `num_entries` пользователь указывает размерность массива `devices`. Для получения массива устройств разработчику необходимо указать платформу `platform` и тип запрашиваемых устройств с помощью константы `device_type`, которая может быть равна следующим значениям.

`CL_DEVICE_TYPE_CPU` - процессоры

`CL_DEVICE_TYPE_GPU` - видеокарты

`CL_DEVICE_TYPE_ACCELERATOR` - ускорители, поддерживающие OpenCL

`CL_DEVICE_TYPE_CUSTOM` - ускорители, не поддерживающие OpenCL

`CL_DEVICE_TYPE_ALL` - все устройства, поддерживающие OpenCL

`cl_int clGetDeviceInfo (cl_device_id device, cl_device_info param_name, size_t param_value_size, void *param_value, size_t *param_value_size_ret)`

Функция предоставляет информацию об устройстве `device`, занося ее по адресу указателя `param_value`, по указателю `param_value_size_ret`

функция заносит размер в байтах значения, занесенного в param_value.

param_value_size - указываемая пользователем размерность значения, чей адрес указан в param_value, device - устройство о котором запрашивается информация, param_name - константа, определяющая какую именно информацию об устройстве запрашивает пользователь, она может принимать значение из достаточно большого диапазона вариантов (для просмотра всех значений смотрите документацию OpenCL).

CL_DEVICE_AVAILABLE (тип cl_bool) - доступно ли устройство

CL_DEVICE_GLOBAL_MEM_CACHE_SIZE (тип cl_ulong) - размер памяти кэша устройства в байтах

CL_DEVICE_GLOBAL_MEM_SIZE (тип cl_ulong) - размер глобальной памяти устройства в байтах

CL_DEVICE_MAX_CLOCK_FREQUENCY (тип cl_uint) - максимальная тактовая частота устройства в МГц

CL_DEVICE_MAX_COMPUTE_UNITS (тип cl_uint) - число вычислительных ядер устройства

CL_DEVICE_MAX_WORK_GROUP_SIZE (тип size_t) - максимальное число рабочих элементов в рабочей группе, выполняемой на одном вычислительном модуле (ядре)

CL_DEVICE_PROFILING_TIMER_RESOLUTION (тип size_t) - разрешение таймера устройства в наносекундах

CL_DEVICE_NAME (тип char[]) - имя устройства

CL_DEVICE_EXTENSIONS (тип char[]) - расширенные имена устройства

CL_DEVICE_OPENCL_C_VERSION (тип char[]) - версии OpenCL, поддерживаемые устройством

CL_DEVICE_VENDOR (тип char[]) - наименование производителя устройства

Узнаем количество и список OpenCL устройств на используемой ЭВМ. Для этого в вышеописанную программу, после цикла вывода информации об OpenCL платформах и ДО очищения памяти от массива платформ добавим следующий программный код.

```
cl_uint numd; //число OpenCL устройств
err=clGetDeviceIDs(platforms[0], CL_DEVICE_TYPE_ALL, 0, 0,
&numd);
if ( err == CL_SUCCESS && numd!=0 )
printf("\n\nОпределено %d OpenCL устройств.\n", numd);
```

```

else
{ printf("\n\nОшибка вызова clGetDeviceIDs или отсутствуют OpenCL
устройства.\n"); return -1 ; }
//массив OpenCL устройств
cl_device_id *devices = new cl_device_id [numd] ;
//получение массива устройств
err=clGetDeviceIDs(platforms[0],  CL_DEVICE_TYPE_ALL,  numd,
devices, 0);

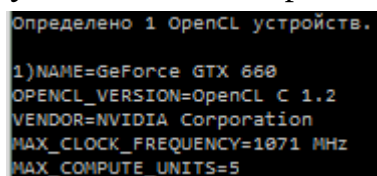
cl_uint arg_uint ;
for (int i=0 ; i<numd ; i++) //цикл обхода всех устройств
{   printf("\n%d",i+1) ;
    clGetDeviceInfo(devices[i], CL_DEVICE_NAME, 250, (void *)str,
0);

    printf("NAME=%s",str) ;
    clGetDeviceInfo(devices[i],
CL_DEVICE_OPENCL_C_VERSION, 250, (void *)str, 0);
    printf("\nOPENCL_VERSION=%s",str) ;
    clGetDeviceInfo(devices[i], CL_DEVICE_VENDOR, 250, (void
*)str, 0);
    printf("\nVENDOR=%s",str) ;
    clGetDeviceInfo(devices[i],
CL_DEVICE_MAX_CLOCK_FREQUENCY,  sizeof(arg_uint),  (void  *)
&arg_uint, 0);
    printf("\nMAX_CLOCK_FREQUENCY=%d MHz",arg_uint) ;
    clGetDeviceInfo(devices[i],
CL_DEVICE_MAX_COMPUTE_UNITS, sizeof(arg_uint), (void *) &arg_uint,
0);
    printf("\nMAX_COMPUTE_UNITS=%d",arg_uint) ;
}

delete [numd] devices ; //очищение памяти от массива устройств

```

Запустим программу и увидим список OpenCL устройств (рис. 6).



```

Определено 1 OpenCL устройств.
1)NAME=GeForce GTX 660
OPENCL_VERSION=OpenCL C 1.2
VENDOR=NVIDIA Corporation
MAX_CLOCK_FREQUENCY=1071 MHz
MAX_COMPUTE_UNITS=5

```

Рис. 6. Выполнение программы, выводящей список OpenCL устройств

Как видно из рис. 6, на конкретной ЭВМ было найдено только одно OpenCL устройство - видеокарта GeForce GTX 660, занесенное в ячейку массива `devices[0]`.

3. Создание контекста

Контекст описывает параметры, необходимые для дальнейшего запуска и выполнения OpenCL программы.

```
cl_context clCreateContext(const cl_context_properties *properties,  
                           cl_uint num_devices, const cl_device_id *devices,  
                           (void CL_CALLBACK *pfn_notify) (const char *errinfo, const void  
*private_info, size_t cb, void *user_data),  
                           void *user_data, cl_int *errcode_ret)
```

Функция создает и возвращает контекст с указанными параметрами.

`properties` - указатель на массив свойств контекста, куда включается платформа `CL_CONTEXT_PLATFORM` типа `cl_platform_id`, а также ряд других записей (см. документацию OpenCL). Массив `properties` может содержать одно или несколько свойств, при описании свойства сначала указывается его константа (например, `CL_CONTEXT_PLATFORM`), затем его значение, взятое с приведением его к типу `cl_context_properties`. Последний элемент массива `properties` приравнивается нулю. (см. пример программного кода ниже) Указатель `properties` можно указать как `NULL`, тогда свойства контекста система выберет по-умолчанию.

`num_devices` - количество устройств, включаемых в контекст.

`devices` - указатель на массив устройств, включаемых в контекст.

`pfn_notify` - указатель на функцию обратного вызова OpenCL типа `void`, которая будет выполнена при возникновении ошибки, связанной с контекстом, аргументы этой функции `errinfo`, `private_info`, `cb`, `user_data` передают в функцию информацию об ошибке. `pfn_notify` можно указать как `NULL` (0).

`user_data` - указатель на пользовательские данные, передаваемые в одноименный аргумент функции `pfn_notify`

`errcode_ret` - указатель, по которому функцию заносит информацию о возможной ошибке вызова `clCreateContext`. По адресу `errcode_ret` функция может записать следующие значения: `CL_SUCCESS` - функция выполнялась успешно, контекст создан, либо ошибки `CL_INVALID_PLATFORM`, `CL_INVALID_PROPERTY`, `CL_INVALID_VALUE`, `CL_INVALID_DEVICE`, `CL_DEVICE_NOT_AVAILABLE`, `CL_OUT_OF_RESOURCES` и др.

Добавим в код вышеописанной программы после определения платформ и устройств и ДО очищения памяти от них следующий код.

```
//массив свойств контекста
cl_context_properties properties[3] =
{CL_CONTEXT_PLATFORM, (cl_context_properties)platforms[0], 0};
//занесли в свойства контекста нулевую (первую найденную)
платформу

cl_int arg_int ;
//создание контекста context
cl_context context = clCreateContext(properties, numd, devices, NULL,
NULL, &arg_int) ; //передали в контекст платформу (через свойства
properties)
//и массив OpenCL устройств devices количеством numd
if (arg_int != CL_SUCCESS)
{ printf("\n\nОшибка создания контекста") ; return -1 ; }
else printf("\n\nКонтекст создан") ;
```

Запустим программу и убедимся, что контекст создается.

4. Объявление очереди команд

Очередь команд связывает выполняющуюся программу с конкретным устройством (только одним) и определяет ряд дополнительных свойств ее будущего исполнения. Если в наличии имеется несколько OpenCL устройств, можно связать с каждым из них свою очередь команд, и таким образом, задействовать их для параллельного выполнения нескольких подпрограмм.

cl_command_queue clCreateCommandQueue(cl_context context, cl_device_id device, cl_command_queue_properties properties, cl_int *errcode_ret)

Функция создает и возвращает очередь команд типа cl_command_queue для указанного контекста context и устройства device. В errcode_ret заносится код ошибки либо CL_SUCCESS в случае успешного выполнения функции. Свойства properties можно указать нулем или подключить в них битовые поля: CL_QUEUE_PROFILING_ENABLE - включить профилирование команд в очереди, CL_QUEUE_OUT_OF_ORDER_EXEC_MODE_ENABLE - включить параллельное выполнение нескольких ядер (понятие ядро будет раскрыто ниже, если такой параметр не указан, ядра будут выполняться последовательно в

порядке их вызова).

Добавим в код вышеописанной программы после определения платформ, устройств, контекста и ДО очищения памяти следующий код.

```
//ОЧЕРЕДЬ КОМАНД для первого найденного OpenCL устройства
cl_command_queue command_queue = clCreateCommandQueue(context,
devices[0], 0, &arg_int);
if (arg_int != CL_SUCCESS)
{ printf("\nОшибка создания очереди команд") ; return -1 ; }
else printf("\nОчередь команд создана") ;
```

Запустим программу и убедимся, что очередь команд создается.

5. Задание программного кода для OpenCL устройства

Программный код, создаваемый для выполнения на OpenCL устройстве, объявляется в основном программном коде как совокупность строк текста (или может храниться в отдельном текстовом файле, откуда его также понадобится считывать, например, в массив строк или в двумерный массив типа char). Эта особенность позволяет в перспективе программно (т.е. динамически, во время выполнения программы на CPU) генерировать код для выполнения на OpenCL устройстве.

В программе объявим строку, содержащую программный код для выполнения на OpenCL устройстве.

```
const char *opengl_source = "__kernel void Simple (__global int *n)
{*n=(*n)*2 ; } \n" ;
```

Можно вывести ее содержимое на экран командой printf ("\n\n%s", opengl_source).

В строке opengl_source текст "__kernel" - это двойное подчеркивание kernel (термин __kernel как раз определяет функцию-ядро, т.е. функцию с которой начнется выполнение команд на OpenCL устройстве). Здесь в текстовой форме задана функция Simple, получающая в аргументе указатель на int и умножающая его значение на два.

6. Создание программного объекта и его компиляция

Программный объект включает текст OpenCL программы, а также ряд других параметров.

cl_program clCreateProgramWithSource (cl_context context, cl_uint count, const char **strings, const size_t *lengths, cl_int *errcode_ret)

Функция создает и возвращает программный объект cl_program для указанного контекста context, текст программы назначается через strings, count - число строк текста в strings, lengths - одномерный массив, хранящий

длину каждой строки в strings, если все строки в strings завершаются символом '\n' (а в пункте 5 строка `opencl_source` описана именно так), то `lengths` можно указать как `NULL(0)`, по адресу `errcode_ret` заносится код ошибки либо `CL_SUCCESS` в случае успешного выполнения функции.

Примечание. Программный объект можно создавать и из готового бинарного исполняемого файла OpenCL с помощью функции **`clCreateProgramWithBinary`**, что может быть полезно при длительном времени срабатывания нижеописанной функции компиляции и построения OpenCL программы `clBuildProgram`.

`cl_int clGetProgramInfo (cl_program program, cl_program_info param_name, size_t param_value_size, void *param_value, size_t *param_value_size_ret)`

Функция возвращает по адресу `param_value` информацию о программном объекте `program` указанного типа `param_name` - эта константа может принимать следующие значения.

`CL_PROGRAM_CONTEXT` (тип `cl_context`) - контекст.

`CL_PROGRAM_REFERENCE_COUNT` (тип `cl_uint`) - число строк.

`CL_PROGRAM_SOURCE` (тип `char[]`) - текст исходного кода.

`CL_PROGRAM_NUM_DEVICES` (тип `cl_uint`) - количество устройств, ассоциированных с программой.

`CL_PROGRAM_DEVICES` (тип `cl_device_id[]`) - массив устройств, ассоциированных с программой.

Следующие параметры будут доступны после построения (см. функцию ниже) программного объекта.

`CL_PROGRAM_BINARY_SIZES` (тип `size_t[]`) - массив размера в байтах исполняемого двоичного файла для каждого ассоциированного с программой устройства отдельно.

`CL_PROGRAM_BINARIES` (тип `unsigned char *[]`) - массив исполняемых двоичных файлов, для всех устройств, связанных с программой.

`CL_PROGRAM_NUM_KERNELS` (тип `size_t`) - число ядер.

`CL_PROGRAM_KERNEL_NAMES` (тип `char[]`) - массив имен функций-ядер.

`param_value_size` - указываемая пользователем размерность значения, чей адрес указан в `param_value`. По адресу `param_value_size_ret` функция заносит размер в байтах значения, реально занесенного в `param_value`.

`cl_int clBuildProgram (cl_program program, cl_uint num_devices,`

**const cl_device_id *device_list, const char *options, void
(CL_CALLBACK *pfn_notify) (cl_program program, void *user_data),
void *user_data)**

Функция компилирует программу *program* и собирает бинарный исполняемый файл. (Для отдельной компиляции и сборки можно применять функции `clCompileProgram` и `clLinkProgram`.) Функция возвращает код ошибки или `CL_SUCCESS` в случае успешного выполнения.

Компиляция и сборка производятся для числа устройств `num_devices`, массив этих устройств указывается в *device_list*.

options - это указатель на строку текста с опциями для компиляции и сборки, строка опций должна завершаться символом '\n'. Если *options* указан как `NULL`, используются настройки по-умолчанию. Список возможных опций обширен (см. документацию на khronos.org), приведем некоторые из них.

-cl-opt-disable - отключить оптимизацию, по-умолчанию оптимизация включена.

-w - отключить предупреждающие сообщения (warning).

-Werror - предупреждающие сообщения получают статус предупреждений об ошибке (error).

pfn_notify - указатель на функцию обратного вызова. Указанная функция будет вызвана по факту выполнения действий функции `clBuildProgram`, успешных или неуспешных. Вызываемая функция обязана иметь список аргументов (*cl_program program, void *user_data*). Можно указать *pfn_notify* как `NULL`.

Последний аргумент *user_data* - это указатель на пользовательские данные, передаваемые в функцию обратного вызова *pfn_notify*, *user_data* можно указать как `NULL`.

Добавим в код вышеописанной программы после определения платформ, устройств, контекста, очереди команд и ДО очищения памяти следующий код.

```
//создание программного объекта из одной (1) строки opencl_source
cl_program opencl_program = clCreateProgramWithSource
(context, 1, &opencl_source, NULL, &arg_int) ;
if (arg_int != CL_SUCCESS)
{ printf("\nОшибка создания программного объекта") ; return -1 ; }
else printf("\nПрограммный объект создан") ;
```

```

        //компиляция и сборка программного объекта для 1 OpenCL
устройства
        err=clBuildProgram (opencl_program, 1, devices, NULL, NULL, NULL)
;
        if ( err != CL_SUCCESS )
        { printf("\nОшибка компиляции или сборки программы") ; return -1 ; }
        else printf("\nПрограмма скомпилирована, создан исполняемый
файл") ;

```

Запустим программу и убедимся, что программный объект создается, компилируется и собирается.

Таким образом, компиляция и сборка OpenCL программы происходят динамически во время выполнения основной программы в режиме "черного ящика", т.е. логи компилятора и сборщика OpenCL разработчик не видит. Для их просмотра будет полезен следующий вызов.

cl_int clGetProgramBuildInfo (cl_program program, cl_device_id device, cl_program_build_info param_name, size_t param_value_size, void *param_value, size_t *param_value_size_ret)

Функция заносит по адресу *param_value* информацию о процессе построения программы *program* для устройства *device*.

param_value_size - указываемая пользователем размерность значения, чей адрес указан в *param_value*. По адресу *param_value_size_ret* функция заносит размер в байтах значения, реально занесенного в *param_value*.

param_name - константа, определяющая, какую именно информацию запрашивает пользователь, она может принимать следующие значения.

CL_PROGRAM_BUILD_STATUS (тип *cl_build_status*) - константа, описывающая статус построения программы: *CL_BUILD_NONE* - построение не произошло, *CL_BUILD_IN_PROGRESS* - построение в процессе, *CL_BUILD_ERROR* - при построении произошла ошибка, *CL_BUILD_SUCCESS* - построение прошло успешно.

CL_PROGRAM_BUILD_OPTIONS (тип *char[]*) - строка опций сборщика.

CL_PROGRAM_BUILD_LOG (тип *char[]*) - строка с логом сборщика.

Улучшим часть программного кода, в котором происходило построение OpenCL программы, а именно, выведем на экран лог сборщика и сделаем это через функцию обратного вызова, которую свяжем с вызовом *clBuildProgram*. (Если OpenCl программа собрана успешно, лог сборщика не будет содержать текста.)

Для этого прошлый вызов.

```
err=clBuildProgram (openccl_program, 1, devices, NULL, NULL, NULL)
;
```

Заменим на вызов.

```
err=clBuildProgram (openccl_program, 1, devices, NULL,
                    func_cl_program, (void *)devices) ;
//подключаем пока еще не описанную функцию обратного вызова
//func_cl_program и передаем ей через void* указатель на массив
//устройств, как помним, в этом примере в нем будет одно
устройство
```

До функции main объявим функцию обратного вызова func_cl_program.

```
void __stdcall func_cl_program (cl_program program, void *user_data)
{
    printf("\n\tСобытие clBuildProgram") ;
    //получение параметра типа cl_device_id из void*
    //т.к. в примере устройство одно, то
    //заносим его в одну переменную, а не в массив
    cl_device_id device = *(cl_device_id *)user_data ;

    char str[2000] ;    cl_int err ;
    err=clGetProgramBuildInfo (program, device,
        CL_PROGRAM_BUILD_LOG, 2000, (void *)str, NULL) ;
    if ( err != CL_SUCCESS )
    { printf("\n\tОшибка вызова clGetProgramBuildInfo") ; return ; }
    //если строка лога сборщика пустая, выйдем из функции
    if (str[0]!='\0' || str[0]!='\n') return ;
    printf("\n\tЛОГ OpenCL СБОРЩИКА:");
    printf("\n%s\n",str);
}
```

Запустим программу и проверим ее выполнение. Если у вас нет ошибок сборки, можно намеренно внести ошибку в строку с текстом OpenCL программы openccl_source и рассмотреть лог OpenCL сборщика (рис. 7).

Рис. 7. Выполнение программы, выводящей лог OpenCL сборщика

7. Создание объекта ядра

Объект ядра содержит функцию-ядро (`__kernel`, это функция, с которой начинается выполнение команд на GPU), а также ее аргументы.

`cl_kernel clCreateKernel (cl_program program, const char *kernel_name, cl_int *errcode_ret)`

Функция создает и возвращает объект-ядра, ассоциированного с именем функции-ядра `kernel_name`, хранящейся в программном объекте `program`. По адресу `errcode_ret` функция записывает код ошибки или `CL_SUCCESS` в случае успешного выполнения.

Значения аргументов ядра задаются отдельной функцией, а именно.

`cl_int clSetKernelArg (cl_kernel kernel, cl_uint arg_index, size_t arg_size, const void *arg_value)`

Функция для указанного ядра `kernel` назначает аргументу под номером `arg_index` (нумерация аргументов с нуля) значение, хранящееся по адресу `arg_value` (обычно это объект памяти, см. ниже), в `arg_size` указывается размер назначаемого значения в байтах, если это базовый тип, или размер типа объекта памяти. Функция возвращает код ошибки или `CL_SUCCESS`.

`cl_int clGetKernelInfo (cl_kernel kernel, cl_kernel_info param_name, size_t param_value_size, void *param_value, size_t *param_value_size_ret)`

Функция возвращает информацию об объекте ядра `kernel`. Тип запрашиваемой информации конкретизируется через константу `param_name`, которая может принимать следующие значения.

`CL_KERNEL_FUNCTION_NAME` (тип `char[]`) - имя функции-ядра.

`CL_KERNEL_NUM_ARGS` (тип `cl_uint`) - число аргументов ядра.

`CL_KERNEL_ATTRIBUTES` (тип `char[]`) - строка с атрибутами ядра.

`CL_KERNEL_CONTEXT` (тип `cl_context`) - контекст ядра.

`CL_KERNEL_PROGRAM` (тип `cl_program`) - программный объект ядра.

Запрашиваемое значение заносится по адресу `param_value`, через `param_value_size` разработчик задает размер `param_value` в байтах, по адресу `param_value_size_ret` функция заносит реальный размер в байтах значения, занесенного в `param_value`.

Функция возвращает код ошибки или `CL_SUCCESS`.

Добавим в код вышеописанной программы после определения программного объекта и ДО очищения памяти следующий код.

```
cl_kernel kernel = clCreateKernel (opencl_program, "Simple", &arg_int) ;  
if (arg_int != CL_SUCCESS)  
{ printf("\nОшибка создания объекта ядра") ; return -1 ; }  
else printf("\nОбъект ядра создан") ;
```

Запустим программу и убедимся, что объект ядра создается.

Функция `clSetKernelArg` будет применена позднее.

8. Описание объектов памяти

Объект памяти описывает область памяти на устройстве. При выполнении вычислений на GPU, оперативная память ПК недоступна, и наоборот, при вычислениях на CPU недоступна память видеокарты. Таким образом, объекты памяти будут необходимы для передачи данных из оперативной памяти в память GPU перед запуском функции-ядра на GPU, а также для передачи данных (результатов) из памяти GPU в оперативную память после завершения выполнения функции-ядра.

`cl_mem clCreateBuffer (cl_context context, cl_mem_flags flags, size_t size, void *host_ptr, cl_int *errcode_ret)`

Функция создает и возвращает объект памяти (типа буфер). По адресу `errcode_ret` функция записывает код ошибки или `CL_SUCCESS` в случае успешного выполнения. Объект памяти создается в рамках контекста `context`, под него выделяется `size` байт. Через указатель `host_ptr` можно указать адрес ранее выделенного буфера, если назначить его как `NULL`, то данная опция не используется. Значение `flags` задает свойства объекта памяти, он может включать следующие битовые поля.

`CL_MEM_READ_WRITE` - доступен ядру для чтения и записи

`CL_MEM_WRITE_ONLY` - доступен ядру только для записи

`CL_MEM_READ_ONLY` - доступен ядру только для чтения

`CL_MEM_HOST_WRITE_ONLY` - доступен хосту (т.е. базовой программе, изначально запускаемой на CPU) только для записи

`CL_MEM_HOST_READ_ONLY` - доступен хосту только для чтения

`CL_MEM_HOST_NO_ACCESS` - не доступен хосту

А также ряд других битовых полей, необходимых при использовании `host_ptr!=NULL`.

Для совместного решения задачи с применением и CPU (а запускается программа всегда на CPU) и GPU необходимо переносить данные между оперативной памятью (CPU) и памятью OpenCL устройства (обычно это GPU). Для этого существуют следующие OpenCL функции.

`cl_int clEnqueueWriteBuffer (cl_command_queue command_queue,`

cl_mem *buffer*, **cl_bool** *blocking_write*, **size_t** *offset*, **size_t** *size*, **const void** **ptr*, **cl_uint** *num_events_in_wait_list*, **const cl_event** **event_wait_list*, **cl_event** **event*)

Функция копирует данные из *ptr* объемом *size* байт в объект памяти (буфер) *buffer*. Операция задействует очередь команд *command_queue*. Если флаг *blocking_write* равен CL_TRUE, то функция не вернет управление в вызвавшую ее программу, пока копирование данных не завершится, а если равен CL_FALSE, вызвавшая программа приступит к выполнению следующей команды не дожидаясь факта окончания копирования данных. В *offset* указывается смещение в байтах, начиная с которого в буфер будет производится запись (0 соответствует началу буфера). Три последних параметра связаны с более детальным управлением очередью команд и пока не рассматриваются, их можно указать как NULL(0), если они не применяются. Функция возвращает код ошибки или CL_SUCCESS.

cl_int **clEnqueueReadBuffer** (**cl_command_queue** *command_queue*, **cl_mem** *buffer*, **cl_bool** *blocking_write*, **size_t** *offset*, **size_t** *size*, **void** **ptr*, **cl_uint** *num_events_in_wait_list*, **const cl_event** **event_wait_list*, **cl_event** **event*)

Функция считывает данные объема *size* байт из буфера *buffer*, начиная со смещения *offset*, и записывает их в область памяти *ptr*. Смысл других параметров такой же, как у функции **clEnqueueWriteBuffer**.

Добавим в код вышеописанной программы после определения объекта ядра и ДО очищения памяти следующий код.

```
//создаем объект памяти на GPU для хранения одного значения типа
int
cl_mem      mem_for_GPU      =      clCreateBuffer(context,
CL_MEM_READ_WRITE, sizeof(int), NULL, &arg_int) ;
if (arg_int != CL_SUCCESS)
{ printf("\nОшибка создания объекта памяти") ; return -1 ; }
else printf("\nОбъект памяти создан") ;

int mem_for_CPU = 10 ; //значение типа int в оперативной памяти
//копируем один int из оперативной памяти в память GPU
err  =  clEnqueueWriteBuffer  (command_queue,  mem_for_GPU,
CL_TRUE, 0, sizeof(int), (const void *)&mem_for_CPU, 0, 0, 0) ;
if ( err  !=  CL_SUCCESS  ) { printf("\nОшибка вызова
clEnqueueWriteBuffer (mem_for_CPU->mem_for_GPU)"); return -1 ; }
```

Запустим программу и убедимся, что все операции проходят без ошибок.

9. Запуск ядра

Перед запуском функции-ядра на OpenCL устройстве ядру назначают аргументы с помощью вышеописанной функции **clSetKernelArg**.

```
cl_int    clEnqueueNDRangeKernel    (cl_command_queue    command_queue,  
    cl_kernel    kernel,    cl_uint    work_dim,    const    size_t  
    *global_work_offset,    const    size_t    *global_work_size,    const    size_t  
    *local_work_size,    cl_uint    num_events_in_wait_list,    const    cl_event  
    *event_wait_list, cl_event *event)
```

Функция запускает выполнение функции-ядра на OpenCL устройстве (добавляет соответствующую команду в очередь команд).

command_queue - очередь команд.

kernel - объект ядра.

work_dim - число измерений для трех следующих параметров (*global_work_offset*, *global_work_size*, *local_work_size*), может принимать значение 1, 2 или 3, это означает, что в трех перечисленных аргументах будет браться во внимание первый элемент, пара первых элементов или тройка первых элементов.

global_work_offset - смещение, используемое для расчета глобальных идентификаторов (*global_work_size*) рабочего элемента OpenCL. Можно указать как NULL, что соответствует нулевому смещению {0}, {0,0} или {0,0,0}.

global_work_size - общее число рабочих элементов OpenCL, задается 1, 2 или 3-я числами (измерениями, см. аргумент *work_dim*). Например: {3} задает 3 раб.элемента, {2,15} задает 2*15=30 раб.элементов, {2,3,2} задает 2*3*2 = 12 раб.элементов.

local_work_size - число рабочих элементов OpenCL в одной группе, задается 1, 2 или 3-я числами (измерениями, см. аргумент *work_dim*). Например: {2,3} задает 2*3=6 рабочих элементов в одной группе, если при этом общее число раб.элементов (*global_work_size*) задано, например, через {2,6}, то получим 2 группы по 6 рабочих элементов в каждой, если *global_work_size* = {4,6}, то будет 4 группы. Параметры в *global_work_size* должны быть кратны *local_work_size*. *local_work_size* можно указать как NULL, тогда разбиение раб. элементов по группам произойдет автоматически.

Три последних параметра связаны с более детальным управлением

очередью команд и пока не рассматриваются, их можно указать как NULL(0), если они не применяются. Функция возвращает код ошибки или CL_SUCCESS.

cl_int clFinish (cl_command_queue command_queue)

Функция останавливает выполнение основной программы (main и т.п.) пока все команды в очереди команд command_queue не будут выполнены. На практике запущенное на выполнение ядро нужно дождаться и только после этого обрабатывать его результаты.

Каждый рабочий элемент, принадлежащий любой группе, выполняет одну и ту же функцию-ядро. В самой функции-ядре можно с помощью специальных функций узнавать номера (индексы, идентификаторы) того, какая группа и какой рабочий элемент в этой группе сейчас работает (в примере выше функция-ядро таких опций не имеет).

Одна группа рабочих элементов выполняется на одном вычислительном узле OpenCL устройства, может иметь участки разделяемой (общей) памяти и средства синхронизации. (Примечание. Используя понятия параллельного программирования на CPU можно *приблизительно* воспринимать это так, что одна группа рабочих элементов - это процесс, а внутри него идет дополнительное разбиение на рабочие элементы - потоки (легковесные процессы, нити).)

Количество рабочих элементов OpenCL в одной группе (local_work_size) имеет физическое ограничение, связанное с конкретным применяемым OpenCL устройством. Оно может быть получено через специальную функцию.

cl_int clGetKernelWorkGroupInfo (cl_kernel kernel, cl_device_id device, cl_kernel_work_group_info param_name, size_t param_value_size, void *param_value, size_t *param_value_size_ret)

Функция возвращает информацию о рабочей группе для указанного ядра *kernel* и устройства *device*. Тип запрашиваемой информации конкретизируется через константу *param_name*, которая может принимать следующие значения.

CL_KERNEL_GLOBAL_WORK_SIZE (тип size_t[3]) - максимальные измерения для задания пространства рабочих групп (параметр не актуален для непостроенного ядра, а также актуален не для всех OpenCL устройств).

CL_KERNEL_WORK_GROUP_SIZE (тип size_t) - максимальное количество рабочих элементов в одной рабочей группе.

CL_KERNEL_COMPILE_WORK_GROUP_SIZE (тип size_t[3]) -

заданные разработчиком измерения пространства рабочих групп.

CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE (тип `size_t`) - рекомендованное кратное для количества рабочих элементов в одной группе (если запустить количество рабочих элементов не кратное запрашиваемому значению, это может несколько замедлить вычисления).

CL_KERNEL_LOCAL_MEM_SIZE (тип `cl_ulong`) - количество памяти в байтах, выделяемой ядру.

CL_KERNEL_PRIVATE_MEM_SIZE (тип `cl_ulong`) - минимальное количество памяти в байтах, выделяемой рабочему элементу для выполнения ядра.

Запрашиваемая информация заносится по адресу *param_value*, через *param_value_size* разработчик указывает размерность значения, чей адрес указан в *param_value*, а в *param_value_size_ret* функция записывает реальный размер значения, записанного по адресу *param_value*. Функция возвращает код ошибки или CL_SUCCESS.

Добавим в вышеописанную программу (до освобождения памяти) следующий код.

```
//узнаем характеристики рабочих групп
size_t SIZE[3];
err = clGetKernelWorkGroupInfo (kernel, devices[0],
CL_KERNEL_WORK_GROUP_SIZE, sizeof(size_t), (void *)&SIZE[0],
NULL);
if ( err != CL_SUCCESS )
{ printf("\nОшибка вызова clGetKernelWorkGroupInfo"); return -1 ; }
else printf("\nМаксим-е кол-во рабочих элементов в одной
группе=%d", SIZE[0]) ;

err = clGetKernelWorkGroupInfo (kernel, devices[0],
CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE,
sizeof(size_t), (void *)&SIZE[0], NULL) ;
if ( err != CL_SUCCESS )
{ printf("\nОшибка вызова clGetKernelWorkGroupInfo"); return -1 ; }
else printf("\nРекомендованное кратное для кол-ва рабочих элементов
в группе=%d", SIZE[0]) ;
```

Запустим программу и узнаем эти параметры для применяемого OpenCL устройства (рис. 8).

```

Объект ядра создан
Объект памяти создан
Максим-е кол-во рабочих элементов в одной группе=1024
Рекомендованное кратное для кол-ва рабочих элементов в группе=32

```

Рис. 8. Выполнение программы, выводящей характеристики рабочих групп

Как видим из рис. 8, сколько бы измерений мы не задавали для распределения рабочих элементов в одной рабочей группе (`local_work_size`), общее их количество в одной группе не должно превышать 1024, а также, если число рабочих элементов в одной группе будет кратно 32, то такая группа будет иметь некоторое преимущество во времени выполнения перед прочими группами (это связано с архитектурными особенностями OpenCL устройства - видеокарты).

Однако в рамках текущего примера запустим функцию-ядро на OpenCL устройстве только для одной рабочей группы, содержащей только один рабочий элемент. Для этого добавим в вышеописанную программу (до освобождения памяти) следующий код.

```

//задаем в первый (№0) аргумент ядра участок памяти
mem_for_GPU
//а в нем хранится значение 10 скопированное ранее из mem_for_CPU
err = clSetKernelArg (kernel, 0, sizeof(mem_for_GPU), (void *)
&mem_for_GPU);
if ( err != CL_SUCCESS )
{ printf("\nОшибка вызова clSetKernelArg"); return -1 ; }

//зададим количество {1, 0, 0} с учетом трех измерений
//(если бы задали только {1} здесь это бы не стало ошибкой)
SIZE[0]=1 ; SIZE[1]=0 ; SIZE[2]=0 ;
//в этом примере измерения для групп и для элементов задаются
через
//один и тот же массив SIZE (одна группа, один рабочий элемент)
err = clEnqueueNDRangeKernel (command_queue, kernel, 1, NULL,
SIZE, SIZE, 0, 0, 0);
if ( err != CL_SUCCESS )
{ printf("\nОшибка запуска ядра"); return -1 ; }
else printf("\nЯдро запущено на OpenCL устройстве");

//дождемся завершения выполнения ядра всеми группами и
элементами

```

```
err = clFinish(command_queue) ;
if ( err != CL_SUCCESS )
{ printf("\nОшибка завершения ядра"); return -1 ; }
else printf("\nВычисление ядра завершено OpenCL устройством") ;
```

Запустим программу и убедимся, что все операции прошли успешно.

10. Получение результатов вычислений ядра и освобождение OpenCL ресурсов

В текущем примере функция-ядро получает указатель, по адресу которого хранится 10 и умножает его на 2. После завершения вычислений ядра, чтобы результаты его работы были доступны базовой программе, запущенной на CPU, их надо скопировать из памяти видеокарты в память ПК с помощью вышеописанного вызова *clEnqueueReadBuffer*.

После получения результатов вычислений ядра следует освободить OpenCL ресурсы с помощью следующих функций.

```
cl_int clReleaseCommandQueue(cl_command_queue command_queue)
cl_int clReleaseContext (cl_context context)
cl_int clReleaseDevice (cl_device_id device)
cl_int clReleaseKernel (cl_kernel kernel)
cl_int clReleaseMemObject (cl_mem memobj)
cl_int clReleaseProgram (cl_program program)
```

Функции освобождают соответствующие ресурсы и возвращают CL_SUCCESS в случае успешного выполнения.

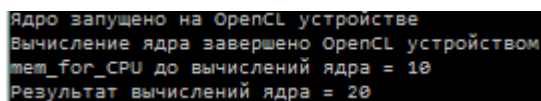
Добавим в вышеописанную программу (до освобождения памяти от массивов устройств и платформ) следующий код.

```
printf("\nmem_for_CPU до вычислений ядра = %d",mem_for_CPU) ;
//получение результатов вычислений ядра
err = clEnqueueReadBuffer (command_queue, mem_for_GPU,
CL_TRUE, 0, sizeof(int), (void *)&mem_for_CPU, 0, 0, 0);
if ( err != CL_SUCCESS )
{ printf("\nОшибка clEnqueueReadBuffer"); return -1 ; }
else printf("\nРезультат вычислений ядра = %d",mem_for_CPU) ;

//освобождение OpenCL ресурсов
clReleaseMemObject(mem_for_GPU);
clReleaseCommandQueue(command_queue);
clReleaseKernel(kernel);
clReleaseProgram(opencl_program);
```

```
clReleaseContext(context);
```

Запустим программу и убедимся, что результат вычислений ядра получен (рис. 9).



```
Ядро запущено на OpenCL устройстве  
Вычисление ядра завершено OpenCL устройством  
mem_for_CPU до вычислений ядра = 10  
Результат вычислений ядра = 20
```

Рис. 9. Выполнение программы, выводящей результат вычислений ядра

Порядок выполнения лабораторной работы

- 1) Разобраться с вышеописанными OpenCL вызовами и терминами.
- 2) Составить и запустить вышеописанную программу.
- 3) Изменить функцию-ядро так, чтобы она обрабатывала не один элемент, а массив элементов. Также запросить у пользователя число/измерения рабочих элементов, запустить функцию-ядро для указанных данных.

Лабораторная работа №2

Тема: Работа с ядром и памятью в OpenCL.

Цели: Изучить особенности памяти OpenCL устройств и принципы распределения задач ядра между рабочими группами и рабочими элементами. Получить навык распараллеливания вычислений на OpenCL устройстве.

Память в OpenCL

Память в модели OpenCL делится на следующие категории.

1) Глобальная память **__global**. Доступна для чтения и записи всем рабочим группам, рабочим элементам, базовой программе на CPU.

2) Константная память **__constant**. Доступна только для чтения всем рабочим группам и рабочим элементам. Доступна для чтения и записи базовой программе на CPU.

3) Локальная память **__local**. Доступна для чтения и записи рабочим элементам, принадлежащим одной рабочей группе (локальная память выделяется каждой рабочей группе отдельно). Базовая программа на CPU не имеет доступа к локальной памяти.

4) Частная память **__private**. Доступна для чтения и записи только одному рабочему элементу (частная память выделяется каждому рабочему элементу отдельно). Базовая программа на CPU не имеет доступа к частной памяти.

Если при описании переменных функции-ядра разработчик не указывает спецификатор (**__global**, **__constant**, **__local**, **__private**), то по умолчанию принимается, что это будет частная память (**__private**). Указатель, объявленный в ядре, может указывать только на адрес в той же памяти, к которой относится сам указатель.

Разделение задач ядра между параллельными примитивами

Обратимся к примеру в предыдущей ЛР. Рис. 6 отображает информацию о наличии 5 вычислительных ядер OpenCL устройства. Это значит, что данное устройство способно параллельно выполнять до 5 рабочих групп.

Каждая рабочая группа состоит из некоторого количества рабочих элементов (по информации на рис. 8 на данном устройстве их общее количество в рамках одной группы не может превысить 1024), которые также способны выполняться параллельно между собою (согласно рис. 8 применяемое устройство параллельно запускает на одном вычислительном

ядре 32 рабочих элемента).

Все рабочие элементы из всех рабочих групп выполняют одну и ту же функцию-ядро. Для назначения рабочим элементам разных подзадач используются функции, возвращающие глобальные (всеобщие) и локальные (в рамках отдельной группы) идентификаторы рабочих элементов и идентификаторы групп.

uint get_work_dim ()

Функция возвращает число измерений, указанное при запуске ядра.

size_t get_global_size (uint dimindx)

Возвращает общее количество рабочих элементов по указанному измерению dimindx. Измерение может принимать значения 0, 1, 2. Например, если всего запускалось {32, 14} рабочих элементов, то get_global_size(0) даст 32, get_global_size(1) даст 14.

size_t get_global_id (uint dimindx)

Возвращает глобальный индекс текущего рабочего элемента по указанному измерению dimindx. Нумерация начинается с нуля. Например, если всего запущено {3} рабочих элемента, то их номера (в измерении 0) 0, 1, 2, а если всего запущено {2,3} рабочих элемента, то их номера {0,0}, {0,1}, {0,2}, {1,0}, {1,1}, {1,2}.

size_t get_local_size (uint dimindx)

Возвращает общее количество рабочих элементов в одной группе по указанному измерению dimindx.

size_t get_local_id (uint dimindx)

Возвращает локальный номер текущего рабочего элемента в текущей группе по указанному измерению dimindx. Нумерация локальных идентификаторов также начинается с нуля.

size_t get_num_groups (uint dimindx)

Возвращает количество групп по указанному измерению dimindx.

size_t get_group_id (uint dimindx)

Возвращает номер текущей группы по указанному измерению dimindx. Нумерация идентификаторов групп также начинается с нуля.

Например, при общем количестве рабочих элементов {6,6} (глобальные идентификаторы) и при заключении в одну группу по {3,3} рабочих элементов (локальные идентификаторы), общее количество групп будет {2,2}=4, а идентификаторы этих групп будут равны {0,0}, {0,1}, {1,0}, {1,1}. Смотрите рис. 10.

size_t get_global_offset (uint dimindx)

Возвращает значение смещения глобального идентификатора рабочего элемента по указанному измерению dimindx.

Глобальное пространство раб.элементов

Группа {0,0}	{0,0}, {0,1}, {0,2}, {0,3}, {0,4}, {0,5}	Группа {0,1}
	{1,0}, {1,1}, {1,2}, {1,3}, {1,4}, {1,5}	
	{2,0}, {2,1}, {2,2}, {2,3}, {2,4}, {2,5}	
Группа {1,0}	{3,0}, {3,1}, {3,2}, {3,3}, {3,4}, {3,5}	Группа {1,1}
	{4,0}, {4,1}, {4,2}, {4,3}, {4,4}, {4,5}	
	{5,0}, {5,1}, {5,2}, {5,3}, {5,4}, {5,5}	

Локальное пространство любой группы =

{0,0}, {0,1}, {0,2}
{1,0}, {1,1}, {1,2}
{2,0}, {2,1}, {2,2}

Рис. 10. Пример распределения рабочих элементов по группам

Пример запуска нескольких рабочих элементов и рабочих групп

Опишем функцию-ядро.

```
const char * opencl_source = "__kernel void Simple (__global int *n)
{ printf(\"\\nI worker! My ID global = {%d, %d}, My ID local = {%d, %d}, My group ID = {%d, %d}\\n\", get_global_id(0), get_global_id(1), get_local_id(0), get_local_id(1), get_group_id(0), get_group_id(1)); } ";
```

Запустим ядро следующим образом.

```
SIZE[0]=4 ; SIZE[1]=4 ; //{4,4} раб. элементов всего
size_t SIZE2[3] = {2,2}; //накуются в группы по {2,2} рабочих элемента
err = clEnqueueNDRangeKernel (command_queue, kernel, 2, NULL, SIZE, SIZE2, 0, 0, 0) ;
```

Запустим программу и рассмотрим идентификаторы рабочих элементов (рис. 11).

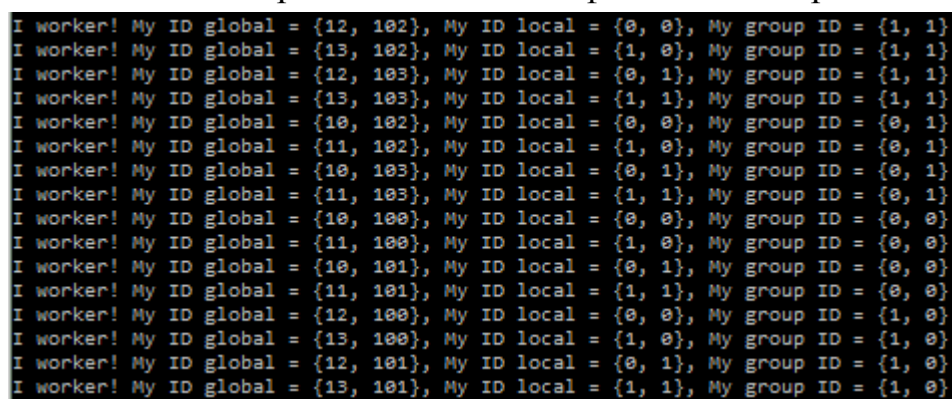
```
I worker! My ID global = {2, 2}, My ID local = {0, 0}, My group ID = {1, 1}
I worker! My ID global = {3, 2}, My ID local = {1, 0}, My group ID = {1, 1}
I worker! My ID global = {2, 3}, My ID local = {0, 1}, My group ID = {1, 1}
I worker! My ID global = {3, 3}, My ID local = {1, 1}, My group ID = {1, 1}
I worker! My ID global = {0, 2}, My ID local = {0, 0}, My group ID = {0, 1}
I worker! My ID global = {1, 2}, My ID local = {1, 0}, My group ID = {0, 1}
I worker! My ID global = {0, 3}, My ID local = {0, 1}, My group ID = {0, 1}
I worker! My ID global = {1, 3}, My ID local = {1, 1}, My group ID = {0, 1}
I worker! My ID global = {2, 0}, My ID local = {0, 0}, My group ID = {1, 0}
I worker! My ID global = {3, 0}, My ID local = {1, 0}, My group ID = {1, 0}
I worker! My ID global = {2, 1}, My ID local = {0, 1}, My group ID = {1, 0}
I worker! My ID global = {3, 1}, My ID local = {1, 1}, My group ID = {1, 0}
I worker! My ID global = {0, 0}, My ID local = {0, 0}, My group ID = {0, 0}
I worker! My ID global = {1, 0}, My ID local = {1, 0}, My group ID = {0, 0}
I worker! My ID global = {0, 1}, My ID local = {0, 1}, My group ID = {0, 0}
I worker! My ID global = {1, 1}, My ID local = {1, 1}, My group ID = {0, 0}
```

Рис. 11. Рабочие элементы OpenCL выводят свои идентификаторы

Рассмотрим как влияет смещение на идентификаторы рабочего элемента. Для этого запустим функцию ядро как описано ниже, с указанием смещения.

```
SIZE[0]=4 ; SIZE[1]=4 ;  
size_t SIZE2[3] = {2,2};  
size_t OFFSET[3] = {10, 100} ; //смещение  
err = clEnqueueNDRangeKernel (command_queue, kernel, 2, OFFSET,  
SIZE, SIZE2, 0, 0, 0) ;
```

Запустим программу и убедимся, что смещение влияет на номера только глобальных идентификаторов рабочих элементов (рис. 12). Это можно будет использовать при описании специфических алгоритмов.



The screenshot displays the output of 14 worker threads. Each line shows the thread's ID, its global ID (offset by 10), its local ID, and its group ID. The global IDs are {12, 13, 12, 13, 10, 11, 10, 11, 10, 11, 10, 11, 13, 12, 13}, which correspond to the values in the OFFSET array {10, 100} repeated for the two dimensions of the kernel. The local IDs are {0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0} and the group IDs are {1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0}.

Рис. 12. Рабочие элементы OpenCL выводят свои идентификаторы со смещением

Пример совместного решения задачи рабочими элементами

Постановка задачи: задан массив вещественных чисел, необходимо возвести каждый элемент массива в квадрат и вычислить сумму возведенных в квадрат элементов.

Здесь будем использовать только одно измерение для определения индексов рабочих элементов, при этом в коде ядра используем только глобальные индексы для распределения подзадач между рабочими элементами.

Рассмотрим один из способов разделения элементов одномерного массива между произвольным числом параллельных примитивов. Сделаем это на конкретном примере, пусть массив на 14 элементов надо распределить между 5 параллельными примитивами. Нулевой примитив изначально ставим в нулевую ячейку, далее двигаемся по массиву с шагом 5. Первый примитив изначально ставим в первую ячейку, далее двигаемся по массиву с шагом 5 (рис.13). И т.д.

№0	№1	№2	№3	№4	№5	№6	№7	№8	№9	№10	№11	№12	№13
0п	1п	2п	3п	4п	0п	1п	2п	3п	4п	0п	1п	2п	3п

Рис. 13. Пример разделения одномерного массива

До создания программы оговоримся, что при наличии глобальной памяти и локальной памяти, общей для всех раб. элементов группы, эта память не защищена от ошибок при перезаписи (**критическая секция**, КС) автоматизированным образом, а механизмы синхронизации и защиты КС в OpenCL мы *пока* не знаем и не применяем. Более того, если в классическом параллельном программировании параллельно работающие потоки (нити) как правило редко способны получить реальную ошибку при выполнении незащищенной критической секции (КС), то при распараллеливании на OpenCL устройстве, ошибка при выполнении КС скорее всего возникнет.

К текущему примеру это имеет такое отношение, раб. элементы должны не только возвести в квадрат элементы массива, но и посчитать их сумму, если они будут производить сложение в одну и ту же переменную в глобальной памяти, то неадекватный финальный ответ будет практически гарантирован.

Сейчас решим вопрос защиты КС так. Заведем массив из подсумм, каждый раб. элемент будет считать свою подсумму в своей ячейке, а после завершения функции-ядра базовая программа найдет общую сумму суммированием этого массива подсумм.

Хотя в дальнейшем будут даны механизмы синхронизации и защиты КС, от подобных подходов к разрешению КС (т.е. защита от КС путем их ликвидации) не стоит отказываться при разработке программ, т.к. упомянутые механизмы несколько повышают время выполнения параллельной программы, и в ряде случаев, "алгоритмическая ликвидация КС" может оказаться выгоднее по времени выполнения программы.

Итак, зададим функцию-ядро.

```
const char * opencl_source =
"__kernel void Kernel(__global float *A, int N, __global float *sum) \n\"
"{ int i, id=get_global_id(0), kol=get_global_size(0);          \n\"
"for (i=id ; i<N ; i=i+kol) { A[i]*=A[i] ; sum[id]+=A[i] ; }      \n\";
```

Массивы A и sum в глобальной памяти, N в частной.

A - массив чисел, N - его размерность, sum - массив подсумм.

id - номер раб. элемента, kol - общее кол-во раб. элементов.

Опишем основные операции в базовой программе.

```
//определение массива вещественных чисел
float F[1024] ; int N=1024 ;
for (int i=0 ; i<N ; i++) F[i]=0 ; F[0]=2 ; F[2]=3 ; F[207]=4 ; F[999]=10 ;
//определение массива из 64 подсумм
float sum[64] ; for (int i=0 ; i<64 ; i++) sum[i]=0 ;
//ОБЪЕКТЫ ПАМЯТИ
//создаем объект памяти на GPU для хранения массива вещ-х чисел
cl_mem      mas_for_GPU      =      clCreateBuffer(context,
CL_MEM_READ_WRITE, N*sizeof(float), NULL, &arg_int) ;
    if (arg_int!=CL_SUCCESS) { printf("\nОшибка создания объекта
памяти mas") ; return -1 ; }
//создаем объект памяти на GPU для хранения 64 вещественных
сумм
cl_mem      sum_for_GPU      =      clCreateBuffer(context,
CL_MEM_READ_WRITE, 64*sizeof(float), NULL, &arg_int) ;
    if (arg_int!=CL_SUCCESS) { printf("\nОшибка создания объекта
памяти sum") ; return -1 ; }

//копируем массивы из оперативной памяти в память OpenCL
устройства
err  =  clEnqueueWriteBuffer (command_queue,  mas_for_GPU,
CL_TRUE, 0, N*sizeof(float), (const void *)F, 0, 0, 0) ;
    if (err!=CL_SUCCESS) { printf("\nОшибка вызова
clEnqueueWriteBuffer mas"); return -1 ; }
err  =  clEnqueueWriteBuffer (command_queue,  sum_for_GPU,
CL_TRUE, 0, 64*sizeof(float), (const void *)sum, 0, 0, 0) ;
    if (err!=CL_SUCCESS) { printf("\nОшибка вызова
clEnqueueWriteBuffer sum"); return -1 ; }

//АРГУМЕНТЫ ЯДРА
//задание нулевого аргумента ядра
err  =  clSetKernelArg (kernel, 0, sizeof(mas_for_GPU), (void *)
&mas_for_GPU) ;
    if (err!=CL_SUCCESS) { printf("\nОшибка вызова clSetKernelArg 0");
return -1 ; }
//задание первого аргумента ядра
```

```

err = clSetKernelArg (kernel, 1, sizeof(int), (void *)&N);
if (err!=CL_SUCCESS) { printf("\nОшибка вызова clSetKernelArg 1");
return -1 ; }

//задание второго аргумента ядра
err = clSetKernelArg (kernel, 2, sizeof(sum_for_GPU), (void
*)&sum_for_GPU);
if (err!=CL_SUCCESS) { printf("\nОшибка вызова clSetKernelArg 2");
return -1 ; }

//ЗАПУСК ЯДРА , всего 64 раб.элемента, поделены на 2 группы
size_t sizeAll=64, sizeGroup=32;
err = clEnqueueNDRangeKernel (command_queue, kernel, 1, NULL,
&sizeAll, &sizeGroup, 0, 0, 0);
if (err!=CL_SUCCESS) { printf("\nОшибка запуска ядра"); return -1 ; }

//дождемся завершения выполнения ядра всеми элементами
err = clFinish(command_queue);
if (err!=CL_SUCCESS) { printf("\nОшибка завершения ядра"); return -
1 ; }

//получение результатов вычислений ядра - МАССИВ
err = clEnqueueReadBuffer (command_queue, mas_for_GPU,
CL_TRUE, 0, N*sizeof(float), (void *)F, 0, 0, 0);
if (err!=CL_SUCCESS) { printf("\nОшибка clEnqueueReadBuffer");
return -1 ; }

//получение результатов вычислений ядра - СУММЫ
err = clEnqueueReadBuffer (command_queue, sum_for_GPU,
CL_TRUE, 0, 64*sizeof(float), (void *)sum, 0, 0, 0);
if (err!=CL_SUCCESS) { printf("\nОшибка clEnqueueReadBuffer");
return -1 ; }

//вывод результатов на экран
float s=0 ; for (int i=0 ; i<64 ; i++) s+=sum[i] ; //сложение всех подсумм
printf("\nСумма=%f \nМассив F:\n",s);
for (int i=0 ; i<N ; i++) {printf("%.1f, ", F[i]) ; if ((i+1)%10==0)
printf("\n") ;}

```

Запустим программу и убедимся в ее работоспособности.

Использование нескольких функций в OpenCL программе

При вычислениях на OpenCL устройстве можно применять несколько функций, а не только одну функцию-ядро (но выполнение OpenCL программы всегда начинается с ядра).

Например, в примере выше можно задать такую OpenCL программу.

```
const char * opencl_source =  
"float F (float x) { return x*sin(x) ; }           \n\"  
"__kernel void Kernel(__global float *A, int N, __global float *sum) \n\"  
"{ int i, id=get_global_id(0), kol=get_global_size(0) ;           \n\"  
"for (i=id ; i<N ; i=i+kol) { A[i]=F(A[i]) ; sum[id]+=A[i] ; } }   \n";
```

Функция *f* работает в частной памяти каждого раб. элемента. Вспомогательные функции и ядро могут быть рекурсивными. Встроенные функции `printf`, `sqrt`, `sin` и т.п. имеют реализацию в библиотеке OpenCL, именно их мы и задействовали во всех примерах. Библиотека OpenCL дополнительно поддерживает упрощенные версии математических функций **native_sqrt**, **native_sin**, **native_cos** и т.п., они имеют меньшую точность и время выполнения.

Порядок выполнения лабораторной работы

1) Разработать OpenCL программу, решающую задачу из вашего варианта. Распределить подзадачи между рабочими элементами таким образом, чтобы загрузка рабочих элементов была равномерна. Для **ввода входных данных задачи и вывода результатов использовать файл(ы)**.

2) Протестировать работу OpenCL программы, убедиться, что она работает корректно.

Варианты

1) Задан одномерный массив целых чисел и его размерность. Найти сумму тех элементов массива, которые делятся нацело на количество двоек в их записи. (Например, число "2292" содержит 3 двойки и $2292/3$ делится нацело и равно 764.)

2) Дан массив целых чисел и его размерность. Вычислить выражение $\pm a_0 \pm a_1 \pm \dots \pm a_n$, в котором со знаком плюс берутся четные элементы массива, а со знаком минус нечетные.

3) Дан массив целых чисел и его размерность. Найти все общие делители этих чисел.

4) Даны одномерные массивы чисел *A* и *B* одинаковой размерности, составить массив *C* той же размерности, записывая в *C[i]* максимальную величину из (*A[i-1]*, *A[i]*, *A[i+1]*, *B[i-1]*, *B[i]*, *B[i+1]*).

5) Задан одномерный численный массив и его размерность. Найти сумму элементов массива, равных тройке в произвольной целой степени.

6) Задано число элементов массива N и число рабочих элементов OpenCL K . Заполнить массив из N элементов элементами последовательности $a_i = (-1)^{i+1} * i * (\sin(i) + \cos^3(i))$.

7) Задан одномерный численный массив и его размерность. Найти все элементы массива, являющиеся квадратами простых чисел.

8) Заданы размерности массивов и сами одномерные массивы целых чисел A и B . Найти сумму тех элементов из A , которые кратны всем элементам из B .

9) Задан одномерный массив целых чисел и его размерность. Составить новый массив той же размерности, каждый элемент которого равен сумме цифр соответствующего элемента из заданного массива.

10) Задан одномерный массив вещественных чисел и его размерность. Вычислить максимальный элемент массива и его индекс.

11) Дана строка символов $C = \{c_0 \dots c_{n-1}\}$. Дан набор из N пар кодирующих символов (a_i, b_i) . Закодировать строку C , заменив в ней все символы a_0 на символы b_0 , все символы a_1 на символы b_1 , и т.д.

12) Дан одномерный массив положительных чисел и его размерность. Каждый элемент массива является длиной стороны куба. Вычислить для каждого куба его объем и площадь поверхности.

13) Даны границы интервала $[a, b]$ и шаг h . Протабулировать на заданном интервале с заданным шагом функцию $f(x) = (x^2 - x - 1)^{1/2}$.

14) Задан одномерный массив целых чисел и его размерность. Найти наибольшее простое число.

15) Задано произвольное число записей вида { ФИО, сумма денежного вклада, начисляемый каждый месяц процент }. Дано натуральное N - количество месяцев. Рассчитать как изменится за указанное число месяцев денежная сумма на каждом счете.

16) Задан одномерный массив целых чисел и его размерность. Для каждого элемента массива найти количество содержащихся в нем цифр.

17) Заданы массивы чисел A и B и их размерность, сформировать массив C , в котором каждое $c_i = \max(a_i, b_i)$.

18) Задан одномерный массив вещественных чисел и его размерность. Вычислить минимальный положительный элемент массива.

19) Заданы размерность и два массива целых чисел одинаковой размерности A и B . Найти сокращенную запись каждой дроби $A[i]/B[i]$.

(Например, для 40/100 сокращенная запись это 2/5.)

20) Задан одномерный массив целых чисел размерности N . Для каждого элемента массива с индексами $[1 \dots N-2]$ проверить делиться ли элемент нацело на сумму пары окружающих его элементов.

21) Задан одномерный массив вещественных чисел и его размерность. Вычислить минимальный элемент массива и его индекс.

22) Заданы натуральное K и вещественное положительное N . Задано произвольное число записей вида { номер телефона, использованный за месяц трафик }, определить для каждого номера, какую сумму должен заплатить абонент за интернет, если K Гб и менее за месяц предоставляются бесплатно, а все что выше округляется и взимается N рублей за 1 Гбайт.

23) Дана строка символов $S = \{c_0 \dots c_{n-1}\}$ и символ b . Определить количество вхождений символа b в первой половине строки S и количество вхождений символа b во второй половине строки S .

24) Даны длины сторон прямоугольника P . Задана размерность и массив положительных чисел, каждый элемент которого это сторона квадрата. Для каждого квадрата определить, сколько таких квадратов может быть размещено на площади прямоугольника P . (Учесть, что например, на прямоугольнике со сторонами 1000 и 0,5 квадрат со стороной = 1 разместиться не может.)

25) Задан одномерный массив вещественных чисел и его размерность. Найти сумму положительных элементов и сумму отрицательных элементов массива.

26) Дано количество и множество телефонных номеров, а также шаблон номера, состоящий из цифр и '*'. Определить все номера, соответствующие шаблону. (Например, шаблону "920*800**77", соответствуют номера "92028002377", "92008001977" и т.п.)

27) Задан одномерный массив вещественных чисел и его размерность. Вычислить произведение положительных элементов массива.

28) Задан одномерный массив целых чисел и его размерность. Заданы целые числа A и B . Для каждого элемента массива найти количество его делителей из интервала $[A, B]$.

29) Задан одномерный массив вещественных чисел и его размерность. Найти количество положительных чисел в массиве и количество отрицательных чисел в массиве.

30) Дано положительное N . Задано множество семян, каждое семя

описано парой { качество в интервале [0%, 100%], предполагаемое число плодов }. Вычислить предполагаемый урожай, если известно, что семя качества ниже N% не прорастет.

31) Задан одномерный массив целых чисел и его размерность. Найти сумму четных чисел и количество нечетных чисел в массиве.

32) Задана размерность массива и сам массив целых чисел A. Составить девять новых массивов той же размерности, записав в первый массив остатки от деления $A[i]$ на 2, во второй массив остатки от деления $A[i]$ на 3, ... в последний массив остатки от деления $A[i]$ на 10.

33) Задан одномерный массив целых чисел и его размерность. Найти произведение простых чисел из массива.

34) Дано произвольное число секторов круга, каждый из которых задан через угол и радиус. Вычислить площадь каждого сектора круга.

35) Задан одномерный массив целых чисел и его размерность. Найти всех элементы, являющиеся квадратами любого натурального числа.

36) Задано множество выражений вида (число1 +,-,/,* число2). Например: $2+3$, $3-6$, $1.2*60$, $5+0.6$ и т.д. Вычислить все выражения.

37) Из известных массивов чисел A и B одинаковой размерности сформировать массив C, в котором каждое c_i равно среднему арифметическому a_i и b_i .

38) В строке произвольной размерности вычислить количество последовательностей повторяющихся символов. (Например для строки "qwweeertt tt" такое количество равно 4, это "ww", "eee", "tt", "ttt".)

39) Задан одномерный массив вещественных чисел A и его размерность. Вычислить выражение $a_0 - a_1 + a_2 - a_3 + a_4 - a_5 + \dots$

40) Дано произвольное число цилиндров, каждый из которых задан через высоту и радиус. Вычислить для каждого цилиндра его объем и площадь поверхности.

41) Дано произвольное число конусов, каждый из которых задан через высоту и радиус. Найти конус с максимальным объемом.

42) Известен радиус окружности S. В массиве положительных чисел заданы стороны квадратов. Для каждого квадрата определить, может ли квадрат быть вписан в окружность S.

43) Задан одномерный массив вещественных чисел и его размерность. Найти произведение всех максимальных и минимальных элементов массива. (Например, для массива {5, 4, 3, 2, 5} такое произведение равно $5*2*5=50$.)

44) Заданы границы интервала $[a, b]$. На заданном интервале приближенно найти длину линии, образованной графиком функцией $f(x) = x^3 + 4x^2 + 2x - 1 - \cos(x)$.

45) Задан одномерный массив целых чисел и его размерность. Вычислить выражение $\pm a_0 \pm a_1 \pm \dots \pm a_n$, в котором со знаком минус берутся элементы массива, являющиеся простыми числами, а со знаком плюс элементы массива, не являющиеся простыми числами.

46) Задан круг S , описанный через координаты центра и радиус. Дано множество точек на плоскости. Определить для каждой точки ее местоположение: внутри круга, вне круга, на дуге круга.

47) Дано натуральное N . Заполнить массив из N элементов элементами последовательности $a_i = i^{i+1} / (i - \sin(i))$.

48) Задан массив, содержащий целые числа и его размерность. Для каждого элемента массива вычислить общее число его четных делителей. (Например, элемент массива = 12, делится на 2, 4 и 6, значит общее число его четных делителей = 3.)

49) Задана размерность и массив положительных чисел, каждый элемент которого это денежная сумма в валюте №1. Задан коэффициент обменного курса. Перевести каждую денежную сумму в валюту №2.

50) Задан одномерный массив вещественных чисел и его размерность. Найти максимальный элемент из элементов с четными индексами и сумму элементов с нечетными индексами.

51) Задан прямоугольник P , описанный через координаты его верхней левой и нижней правой вершин. Дано множество точек на плоскости. Для каждой точки определить, лежит ли точка внутри либо вне прямоугольника.

52) Задано произвольное число результатов бросков кубика игрока1 и аналогичное число результатов бросков кубика игрока2. Определить результат игры: выиграл первый или второй игрок, либо ничья.

53) Заданы натуральные числа N и K , при этом K лежит в границах $[0, 9]$. Задано количество номеров и сами телефонные номера. Найти все номера, содержащие указанную цифру K указанное число раз N .

54) Задан одномерный массив целых чисел и его размерность. Найти произведение элементов, равных двойке в любой целой степени.

55) Задана строка S и строка s . Найти количество вхождений строки s в S . (Например, для $S = \text{"ххууххА"}$ и $s = \text{"хх"}$ количество вхождений = 2.)

56) Задано множество точек на плоскости. Составить одномерный

массив, размерность которого будет совпадать с числом точек. Для каждой точки записать в соответствующую ей ячейку массива, к какой четверти координатной плоскости относится точка.

57) Задан одномерный массив натуральных чисел. Составить новый массив той же размерности, записав в него обратную запись соответствующих элементов изначально заданного массива. (Например, для "9041" обратная запись "1409", для "8400" обратная запись "48".)

58) Задан одномерный массив целых чисел и его размерность. Найти произведение тех чисел из массива, которые не имеют нечетных делителей (единица в роли делителя не учитывается).

59) Задана строка С, содержащая буквы английского алфавита. Вычислить количество гласных букв и количество согласных букв.

60) Задана строка С. Вычислить количество целых чисел, содержащихся в строке. (Не цифр, а чисел. Так строка "123, абывалг 301ша", содержит 6 цифр '1', '2', '3', '3', '0', '1' и 2 числа "123" и "301".)

Лабораторная работа №3

Тема: Синхронизация и защита критических секций.

Цели: Получить навык синхронизации вычислений рабочих элементов и защиты критических секций в OpenCL программе.

Синхронизация. Барьеры

Барьер - распространенный в библиотеках параллельного программирования механизм синхронизации, принуждающий все параллельные примитивы (или некоторую часть из них) дожидаться друг друга в выделенной точке программы.

Есть барьер и в библиотеке OpenCL.

barrier(CLK_LOCAL_MEM_FENCE)

Все рабочие элементы одной рабочей группы ожидают друг друга (т.е. ожидают, чтобы все они произвели данный вызов).

barrier(CLK_GLOBAL_MEM_FENCE)

Все рабочие элементы из всех рабочих групп ожидают друг друга.

Пример использования барьера

В предыдущем примере поиска суммы квадратов заменим функцию-ядро на следующую.

```
const char * openc1_source =
"__kernel void Kernel(__global float *A, int N, __global float *sum) \n\"
"{ int i, id=get_global_id(0), kol=get_global_size(0);          \n\"
"for (i=id ; i<N ; i=i+kol) { A[i]*=A[i] ; sum[id]+=A[i] ; }      \n\"
"barrier(CLK_GLOBAL_MEM_FENCE);                                \n\"
"if (id==0) for (int i=1 ; i<64 ; i++) sum[0]+=sum[i] ; }        \n\";
```

Отличие от предыдущего ядра в добавлении двух последних строк. Здесь происходит следующее, после возведения элементов в квадрат и поиска подсумм все раб. элементы из всех раб. групп ожидают друг друга на барьере (barrier(CLK_GLOBAL_MEM_FENCE)), затем раб. элемент с нулевым номером (if (id==0) for) находит общую сумму и записывает ее в нулевую ячейку массива sum.

Теперь в базовой программе вместо этого кода.

```
float s=0 ; for (int i=0 ; i<64 ; i++) s+=sum[i] ;
printf("\nСумма=%f\nМассив F:\n",s) ;
```

Достаточно записать следующую команду.

```
printf("\nСумма=%f \nМассив F:\n",sum[0]) ;
```

Критические секции

Критическая секция (КС) - это область кода, при выполнении которой несколькими параллельными примитивами, существует вероятность одновременного изменения общего ресурса (переменных, файлов и т.п.) двумя или более параллельными примитивами, что может привести к возникновению ошибки (сбоя).

Пример критической секции

Глобальная (общая) `int i = 0` ; увеличивается в первом и втором потоке на единицу (`i++`), логично полагать, что `i` после завершения работы обоих потоков будет равна двум, однако возможна техническая накладка при возникновении которой `i` примет другое значение. Рассмотрим операции (`i++`) более подробно (табл. 1).

Таблица 1. Выполнение инкремента парой потоков

Поток №1 (<code>i++</code>)	Поток №2 (<code>i++</code>)
1) считать значение <code>i</code> из памяти	1) считать значение <code>i</code> из памяти
2) поместить прочитанное значение в регистр процессора	2) поместить прочитанное значение в регистр процессора
3) увеличить значение в регистре	3) увеличить значение в регистре
4) записать значение из регистра в память <code>i</code>	4) записать значение из регистра в память <code>i</code>

Если, к примеру, первый поток успеет выполнить только пункты 1-3 и второй поток приступит к выполнению пункта 1, он считает из памяти `i` 0, а не 1, и таким образом, оба потока считают нули, увеличат их до единиц, запишут эти единицы в память `i` и финально `i` будет равно единице, а не двойке, после выполнения двух операций `i++`.

Защита критических секций. Атомарные функции

В классическом параллельном программировании защита КС выполняется специальными механизмами: мьютексами, семафорами, блокировками. Эти механизмы имеют отличия между собою, но общий смысл защиты КС одинаков - во избежание ошибок, параллельные примитивы должны выполнять КС последовательно.

В OpenCL перечисленные механизмы отсутствуют. Но зато определено множество атомарных функций, предотвращающих одновременное обращение к памяти. Эти функции разботают только с целочисленными типами, 32-разрядными (`int`, `uint`) или 64-разрядными (`long`, `ulong`), объявленными в глобальной или локальной памяти.

Для использования атомарных функций могут потребоваться нижеописанные директивы. Их подключают в коде OpenCL программы (т.е. в

той же строке(ах), где записана функция-ядро).

```
#pragma OPENCL EXTENSION cl_khr_global_int32_base_atomics :  
enable
```

```
#pragma OPENCL EXTENSION cl_khr_local_int32_base_atomics : ena-  
ble
```

```
#pragma OPENCL EXTENSION cl_khr_global_int32_extended_atomics  
: enable
```

```
#pragma OPENCL EXTENSION cl_khr_local_int32_extended_atomics :  
enable
```

```
#pragma OPENCL EXTENSION cl_khr_int64_base_atomics : enable
```

```
#pragma OPENCL EXTENSION cl_khr_int64_extended_atomics : enable
```

Перечень атомарных функций для 64-разрядных типов.

```
long atom_add (__global long *p, long val)
```

```
long atom_add (__local long *p, long val)
```

```
ulong atom_add (__global ulong *p, ulong val)
```

```
ulong atom_add (__local ulong *p, ulong val)
```

Функция атомарно добавляет к значению по адресу p величину val и возвращает предыдущее значение по адресу p.

Следующие функции имеют аналогичные аргументы и возвращаемые значения: **atom_sub** - вычитание, **atom_and** - и, **atom_or** - или, **atom_xor** - исключающее или, "либо то, либо то, но не оба вместе" {0+0->0, 1+1->0, 0+1->1, 1+0->1}, **atom_max** - максимум, **atom_min** - минимум, **atom_xchg** - приравнивание.

А также инкремент и декремент.

```
long atom_inc (__global long *p), long atom_inc (__local long *p),  
ulong atom_inc (__global ulong *p), ulong atom_inc (__local ulong *p)
```

atom_dec обладает аналогичными аргументами.

Дополнительно:

```
long atom_cmpxchg (__global long *p, long cmp, long val)
```

```
long atom_cmpxchg (__local long *p, long cmp, long val)
```

```
ulong atom_cmpxchg (__global ulong *p, ulong cmp, ulong val)
```

```
ulong atom_cmpxchg (__local ulong *p, ulong cmp, ulong val)
```

Если значение по адресу p равно cmp, оно заменяется на val.

Атомарные функции для 32-разрядных типов также работают с указателями в __global и __local памяти, и с типами int и uint (вместо long и ulong). Принцип их работы аналогичен, имена этих функций: **atomic_add**, **atomic_sub**, **atomic_and**, **atomic_or**, **atomic_xor**, **atomic_max**, **atomic_min**,

atomic_inc, atomic_dec, atomic_xchg, atomic_cmpxchg.

Примеры использования атомарных функций

Обратимся к предыдущему примеру поиска суммы квадратов. В нем работа производилась с вещественными значениями типа float, для этого типа атомарных функций не определено. Если представить, что вычисления производятся над массивом типа int (и сумма этих значений не превышает диапазона типа int), то функция-ядро может иметь такой вид.

```
const char * opencl_source =  
"__kernel void Kernel(__global int *A, int N, __global int *sum)  \n\"  
"{ int i, id=get_global_id(0), kol=get_global_size(0);          \n\"  
"for (i=id;i<N;i=i+kol){A[i]*=A[i]; atomic_add(&sum[0],A[i]); } }  \n\"
```

По окончанию вычислений общая сумма хранилась бы в sum[0], а другие ячейки массива sum остались бы неиспользованными.

Рассмотрим, как можно применить в этом примере атомарные функции если остается тип float.

Сначала рассмотрим принципы объявления переменных в функциях, выполняемых на OpenCL устройстве. Например, при объявлении int x, x выделяется каждому рабочему элементу отдельно. При объявлении __local int x, x выделяется каждой рабочей группе отдельно и общедоступно всем рабочим элементам из этой группы. Динамическое выделение переменной в глобальной памяти, т.е. __global int x, доступно не во всех версиях OpenCL. Зато доступно объявление указателя на глобальную память __global int *x, который впоследствии может указывать на часть глобальной памяти, выделенной перед запуском функции-ядра (clCreateBuffer) и указанной в функцию-ядро через один из ее аргументов.

Применим атомарные функции так: выделим локальную целочисленную сумму для каждой группы (всего их 2, по 32 рабочих элемента в каждой) и каждая группа будет считать свою сумму с помощью атомарных операций (при этом будут потеряны значения после запятой).

```
const char * opencl_source =  
"__kernel void Kernel(__global float *A, int N, __global float *sum) \n\"  
"{ int i, id=get_global_id(0), kol=get_global_size(0);          \n\"  
"__local int SUM; if (id==0 || id==32) SUM=0 ;                  \n\"  
"barrier(CLK_LOCAL_MEM_FENCE);                                \n\"  
"for      (i=id      ;      i<N      ;      i=i+kol)          {  
A[i]*=A[i];atomic_add(&SUM,(int)A[i]);}\n\"  
"barrier(CLK_LOCAL_MEM_FENCE);                                \n\"
```

```
"if (id==0) sum[0]=SUM ; if (id==32) sum[1]=SUM ; }      \n";
```

Здесь происходит следующее, выделяется `int SUM` для каждой группы отдельно, 0-й поток из группы 0 обнуляет свою `SUM`, а 32-й поток из группы 1 обнуляет свою `SUM`, другие потоки ожидают обнуления на барьере. Далее в (каждую из двух) `SUM` происходит атомарное сложение. После синхронизации на барьере, 0-й поток заносит в ячейку массива `sum[0]` сумму, посчитанную группой 0, а 32-й поток заносит в ячейку массива `sum[1]` сумму, посчитанную группой 1.

Теперь в базовой программе меняем оператор вывода результата.

```
printf("\nСумма=%f+%f=%f      \nМассив\n",sum[0],sum[1],sum[0]+sum[1]) ;
```

Порядок выполнения лабораторной работы

1) В программу из ЛР2 добавьте барьеры и атомарные операции там, где это следует из логики варианта ЛР2 и составленной ранее OpenCL программы. Если ваше задание или программа составлены так, что использование барьеров или атомарных операций не требуется, самостоятельно измените задание или программу таким образом, чтобы применение барьеров и атомарных операций стало осмысленным.

2) Протестируйте работу OpenCL программы с барьерами и атомарными операциями, убедитесь, что она выполняется корректно.

Лабораторная работа №4

Тема: Многомерные массивы. Изображения

Цели: Получить навык обработки многомерных массивов и применения специальных типов данных изображений в OpenCL.

Введение

В рамках классического программирования средствами C++ для многомерных массивов (в отличие от одномерных) существуют различия их обработки во вспомогательных функциях в зависимости от того являются эти массивы статическими или динамическими.

Например, память под статический двумерный массив $[N][M]$ выделяется как под одномерный массив $[N*M]$, а при динамическом объявлении двумерного массива, каждая строка выделяется тоже как одномерный массив, но при этом адреса начал этих строк не согласованы в строгой последовательности.

На практике это влияет на правила объявления аргументов и обращений к массиву в функциях. Например, для двумерных массивов.

`f(int **a, int N, int M) { a[i][j]=0; }` - функция обработки двумерного динамического массива $[N][M]$.

`f(int a[N][M]) { a[i][j]=0; }` - функция обработки двумерного статического массива $[N][M]$, где N и M известны и неизменны.

`f(int* arr, int N, int M) { *(arr + i*M + j)=0; }` - функция обработки двумерного статического массива $[N][M]$ или динамического одномерного массива $[N*M]$.

При применении OpenCL, функции работы с памятью таковы, что определяют работу с многомерными массивами по третьему образцу.

Пример OpenCL программы, обрабатывающей двумерный массив

Создадим OpenCL программу, получающую двумерный массив, и повышающую каждый элемент этого массива на единицу. При этом число раб.элементов возьмем равным числу строк массива (т.е. каждый раб. элемент будет обрабатывать свою строку массива), распределение их по группам возложим на саму систему.

Опишем функцию-ядро.

```
const char * opencl_source =
"__kernel void Kernel(__global int *A, int N, int M) \n\"
"{int i=get_global_id(0)*M, j ; for(j=0;j<M;j++) *(A+i+j)=*(A+i+j)+1;}
```

```
\n";
```

Зададим код базовой программы.

```
//определение двумерного массива
int F[64][8] ; int N=64,M=8;
for (int i=0 ; i<N ; i++) for (int j=0 ; j<M ; j++) F[i][j]=0 ;

//создаем объект памяти на GPU для хранения двумерного массива
cl_mem      mas_for_GPU      =      clCreateBuffer(context,
CL_MEM_READ_WRITE, N*M*sizeof(int), NULL, &arg_int) ;
if (arg_int!=CL_SUCCESS) { printf("\nОшибка создания объекта
памяти") ; return -1 ; }

//копируем массив из оперативной памяти в память OpenCL
устройства
err  =  clEnqueueWriteBuffer (command_queue,  mas_for_GPU,
CL_TRUE, 0, N*M*sizeof(int), (const void *)F[0], 0, 0, 0) ;
if (err!=CL_SUCCESS) { printf("\nОшибка вызова
clEnqueueWriteBuffer"); return -1 ; }

//задание нулевого аргумента ядра
err  =  clSetKernelArg (kernel,  0,  sizeof(mas_for_GPU), (void
*)&mas_for_GPU) ;
if (err!=CL_SUCCESS) { printf("\nОшибка clSetKernelArg 0"); return -1
; }

//задание первого аргумента ядра
err = clSetKernelArg (kernel, 1, sizeof(int), (void *)&N) ;
if (err!=CL_SUCCESS) { printf("\nОшибка clSetKernelArg 1"); return -1
; }

//задание второго аргумента ядра
err = clSetKernelArg (kernel, 2, sizeof(int), (void *)&M) ;
if (err!=CL_SUCCESS) { printf("\nОшибка clSetKernelArg 2"); return -1
; }

//ЗАПУСК ЯДРА
size_t sizeAll=N; err = clEnqueueNDRangeKernel (command_queue,
kernel, 1, NULL, &sizeAll, NULL, 0, 0, 0) ;
if (err!=CL_SUCCESS) { printf("\nОшибка запуска ядра"); return -1 ; }
```

```

//дождемся завершения выполнения ядра всеми элементами
err = clFinish(command_queue);
if (err!=CL_SUCCESS) { printf("\nОшибка завершения ядра"); return -
1 ; }

//получение результатов вычислений ядра
err = clEnqueueReadBuffer (command_queue, mas_for_GPU,
CL_TRUE, 0, N*M*sizeof(int), (void *)F[0], 0, 0, 0);
if (err!=CL_SUCCESS) { printf("\nОшибка EnqueueReadBuffer");
return -1;}

//вывод результатов на экран
for (int i=0 ; i<N ; i++) { printf("\n");
for (int j=0 ; j<M ; j++) printf("%d ",F[i][j]) ; }

```

Изображения

Другой способ работы с многомерными массивами состоит в применении специальных типов OpenCL - изображений. Также изображения служат непосредственно для работы с графическими форматами.

Сначала рассмотрим вспомогательный специальный тип **сэмплер sampler_t**, определяющий правила работы с изображением. Сэмплер конкретизирует три параметра.

1) Информация о нормализации координат.

CLK_NORMALIZED_COORDS_TRUE - нормализованные.

CLK_NORMALIZED_COORDS_FALSE - ненормализованные.

В нормализованном значении все разряды числа (включая мантиссу для вещественных значений) заполнены, в ненормализованном значении неиспользованные разряды могут содержать "мусор".

2) Режим адресации определяет поведение при запросе координат, выходящих за пределы границ изображения. Может принимать одно из следующих значений:

CLK_ADDRESS_REPEAT - "копировать" изображение за его пределы (работает вместе с нормализованными координатами).

CLK_ADDRESS_MIRRORED_REPEAT - зеркально "копировать" изображение за его пределы (работает вместе с нормализованными координатами).

CLK_ADDRESS_CLAMP_TO_EDGE - использовать ближайшее

граничное значение.

CLK_ADDRESS_CLAMP - использовать граничное значение.

CLK_ADDRESS_NONE - при этом режиме программист гарантирует, что координаты корректны. Если это не так, результат операции будет не определен.

3) Режим фильтрации определяет поведение при указании не целых координат.

CLK_FILTER_NEAREST - значение по округленным координатам.

CLK_FILTER_LINEAR - линейная интерполяция значения.

Описать сэмплер в программе можно, например, так.

<pre>sampler_t sampler = CLK_NORMALIZED_COORDS_TRUE CLK_ADDRESS_REPEAT CLK_FILTER_NEAREST ;</pre>

Либо через специальную функцию создания сэмплера.

cl_sampler clCreateSampler (cl_context context, cl_bool normalized_coords, cl_addressing_mode addressing_mode, cl_filter_mode filter_mode, cl_int *errcode_ret)

Где context - контекст, normalized_coords - информация о нормализации координат, addressing_mode - режим адресации, filter_mode - режим фильтрации. По адресу errcode_ret функция записывает код ошибки или CL_SUCCESS в случае успешного выполнения.

Также рассмотрим типы данных вида **char_n, int_n, uint_n, long_n, ulong_n, float_n, double_n**, где n может быть равно 2, 3, 4, 8, 16. Например, тип int₄ задает вектор из четырех значений типа int. Для этих типов данных предопределены арифметические операторы (табл. 2), применяются они в коде OpenCL программы, предназначенном для OpenCL устройства.

Таблица 2. Примеры OpenCL программ с векторными типами

Код	Результат выполнения
int4 A = {10,1,15,3}, B={1,1,1,10}, C; C=A+B ; printf("\n%d %d %d %d",C,C+1,C+2,C+3);	11 2 16 13
int4 A = {10,1,15,3}, B= {1,2,3,4}, C; C=A*B ; printf("\n%d %d %d %d",C,C+1,C+2,C+3);	10 2 45 12
int4 A = {10,1,15,3}, B=2, C; C=A*B-5 ; printf("\n%d %d %d %d",C,C+1,C+2,C+3);	15 -3 25 1
int4 A = {10,1,15,3}, B; B.s0=0 ; B.s1=151 ; B.s2=2+3 ; B.s3=A.s0+A.s1 ; printf("\n%d %d %d %d",B.s0,B.s1,B.s2,B.s3);	0 151 5 11

Для типов с размерностью 16, обращение к элементам может производиться через: s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, sA, sB, sC, sD, sE, sF.

Типы изображений

image1d_t - одномерное изображение.

image2d_t - двумерное изображение.

image3d_t - трехмерное изображение.

image1d_array_t - массив одномерных изображений.

image2d_array_t - массив двумерных изображений.

Функции для записи данных в изображения

void write_imagef (image1d_t image, int coord, float4 color)

Записывает в одномерное изображение **image** по координате **coord** цвет **float4 color** (т.е. цвет содержит четыре поля, что применительно к реальным изображениям, как правило, означает запись цвета через RedGreenBlue и коэффициент прозрачности).

void write_imagef (image1d_array_t image, int2 coord, float4 color)

Записывает в массив одномерных изображений **image** по координатам **int2 coord** цвет **float4 color**.

void write_imagei (image1d_t image, int coord, int4 color)

Записывает в одномерное изображение **image** по координате **coord** цвет **int4 color**.

void write_imagei (image1d_array_t image, int2 coord, int4 color)

Записывает в массив одномерных изображений **image** по координатам **int2 coord** цвет **int4 color**.

Далее заголовки подобных функций, работающих с разными типами данных, приводятся без пояснения.

void write_imageui (image1d_t image, int coord, uint4 color)

void write_imageui (image1d_array_t image, int2 coord, uint4 color)

Для двумерных изображений и массивов двумерных изображений.

void write_imagef (image2d_t image, int2 coord, float4 color)

void write_imagef (image2d_array_t image, int4 coord, float4 color)

void write_imagei (image2d_t image, int2 coord, int4 color)

void write_imagei (image2d_array_t image, int4 coord, int4 color)

void write_imageui (image2d_t image, int2 coord, uint4 color)

void write_imageui (image2d_array_t image, int4 coord, uint4 color)

Для трехмерных изображений.

void write_imagef (image3d_t image, int4 coord, float4 color)

void write_imagei (image3d_t image, int4 coord, int4 color)

void write_imageui (image3d_t image, int4 coord, uint4 color)

Директива для обработки трехмерных изображений.

#pragma OPENCL EXTENSION cl_khr_3d_image_writes : enable

Функции для считывания данных из изображений

float4 read_imagef (image1d_t image, sampler_t sampler, int coord)

Функция возвращает значение цвета (float4) из одномерного изображения image в координате coord, при этом применяется сэмплер sampler.

Далее заголовки подобных функций, работающих с разными типами данных, приводятся без пояснения. Обратите внимание, что сэмплер может и не применяться.

float4 read_imagef (image1d_t image, sampler_t sampler, float coord)

float4 read_imagef (image1d_t image, int coord)

float4 read_imagef (image1d_array_t image, int2 coord)

float4 read_imagef (image1d_array_t image, sampler_t sampler, float4 coord)

int4 read_imagei (image1d_t image, sampler_t sampler, int coord)

int4 read_imagei (image1d_t image, sampler_t sampler, float coord)

int4 read_imagei (image1d_t image, int coord)

int4 read_imagei (image1d_array_t image, int2 coord)

int4 read_imagei (image1d_array_t image, sampler_t sampler, int2 coord)

int4 read_imagei (image1d_array_t image, sampler_t sampler, float2 coord)

uint4 read_imageui (image1d_t image, sampler_t sampler, int coord)

uint4 read_imageui (image1d_t image, sampler_t sampler, float coord)

uint4 read_imageui (image1d_t image, int coord)

uint4 read_imageui (image1d_array_t image, int2 coord)

uint4 read_imageui (image1d_array_t image, sampler_t sampler, int2 coord)

uint4 read_imageui (image1d_array_t image, sampler_t sampler, float2 coord)

Для двумерных изображений и массивов двумерных изображений.

float4 read_imagef (image2d_t image, sampler_t sampler, int2 coord)

float4 read_imagef (image2d_t image, sampler_t sampler, float2 coord)

float4 read_imagef (image2d_t image, int2 coord)

float4 read_imagef (image2d_array_t image, int4 coord)

float4 read_imagef (image2d_array_t image, sampler_t sampler, int4 coord)

```

float4 read_imagef (image2d_array_t image, sampler_t sampler, float4
coord)
int4 read_imagei (image2d_t image, sampler_t sampler, int2 coord)
int4 read_imagei (image2d_t image, sampler_t sampler, float2 coord)
int4 read_imagei (image2d_t image, int2 coord)
int4 read_imagei (image2d_array_t image, int4 coord)
int4 read_imagei (image2d_array_t image, sampler_t sampler, int4 coord)
int4 read_imagei (image2d_array_t image, sampler_t sampler, float4
coord)
uint4 read_imageui (image2d_t image, sampler_t sampler, int2 coord)
uint4 read_imageui (image2d_t image, sampler_t sampler, float2 coord)
uint4 read_imageui (image2d_t image, int2 coord)
uint4 read_imageui (image2d_array_t image, int4 coord)
uint4 read_imageui (image2d_array_t image, sampler_t sampler, int4
coord)
uint4 read_imageui (image2d_array_t image, sampler_t sampler, float4
coord)

```

Для трехмерных изображений.

```

float4 read_imagef (image3d_t image, sampler_t sampler, int4 coord)
float4 read_imagef (image3d_t image, sampler_t sampler, float4 coord)
float4 read_imagef (image3d_t image, int4 coord)
int4 read_imagei (image3d_t image, sampler_t sampler, int4 coord)
int4 read_imagei (image3d_t image, sampler_t sampler, float4 coord)
int4 read_imagei (image3d_t image, int4 coord)
uint4 read_imageui (image3d_t image, sampler_t sampler, int4 coord)
uint4 read_imageui (image3d_t image, sampler_t sampler, float4 coord)
uint4 read_imageui (image3d_t image, int4 coord)

```

Внимательно рассматривая множество вышеперечисленных функций можно заметить, что для массива двумерных изображений и для трехмерного изображения координаты описываются типом вида *int4*. Однако реальные координаты - первые три, четвертая недействительна.

Функции, возвращающие параметры изображения.

Возвращают ширину изображения *image*.

```

int get_image_width (image1d_t image)
int get_image_width (image2d_t image)
int get_image_width (image3d_t image)
int get_image_width (image1d_array_t image)

```

int get_image_width (image2d_array_t image)

Возвращают высоту изображения image.

int get_image_height (image2d_t image)

int get_image_height (image3d_t image)

int get_image_height (image2d_array_t image)

Возвращает размерность по третьей оси трехмерного изображения image.

int get_image_depth (image3d_t image)

Возвращают размерности изображений через векторный тип intn (т.е. заносят ширину, высоту, размерность по третьей оси (глубину) в одну переменную).

int2 get_image_dim (image2d_t image)

int2 get_image_dim (image2d_array_t image)

int4 get_image_dim (image3d_t image)

Возвращают количество изображений в массивах.

size_t get_image_array_size (image2d_array_t image)

size_t get_image_array_size (image1d_array_t image)

Возвращают тип хранимых в изображении данных.

int get_image_channel_data_type (image1d_t image)

int get_image_channel_data_type (image2d_t image)

int get_image_channel_data_type (image3d_t image)

int get_image_channel_data_type (image1d_array_t image)

int get_image_channel_data_type (image2d_array_t image)

Возвращаемое значение является одной из следующих констант (таблица 3).

Таблица 3. Константы типов данных изображения

Константа типа	Тип ячеек изображения
CL_SNORM_INT8	Нормализованное знаковое 8-битное целочисленное значение
CL_SNORM_INT16	Нормализованное знаковое 16-битное целочисленное значение
CL_UNORM_INT8	Нормализованное беззнаковое 8-битное целочисленное значение
CL_UNORM_INT16	Нормализованное беззнаковое 16-битное целочисленное значение
CL_UNORM_SHORT_565	Нормализованное 3-х канальное (RGB) изображение. Совмещается с параметром CL_RGB или CL_RGBx (см. таблицы 4 и 5).
CL_UNORM_SHORT_555	Нормализованное 4-х канальное (RGBx) изображение.

	Совмещается с параметром CL_RGB или CL_RGBx (см. таблицы 4 и 5).
CL_UNORM_INT_101010	Нормализованное 4-х канальное (RGBx) изображение. Совмещается с параметром CL_RGB или CL_RGBx (см. таблицы 4 и 5).
CL_SIGNED_INT8	Ненормализованное знаковое 8-битное целочисленное значение
CL_SIGNED_INT16	Ненормализованное знаковое 16-битное целочисленное значение
CL_SIGNED_INT32	Ненормализованное знаковое 32-битное целочисленное значение
CL_UNSIGNED_INT8	Ненормализованное беззнаковое 8-битное целочисленное значение
CL_UNSIGNED_INT16	Ненормализованное беззнаковое 16-битное целочисленное значение
CL_UNSIGNED_INT32	Ненормализованное беззнаковое 32-битное целочисленное значение
CL_HALF_FLOAT	16- битное half-float значение
CL_FLOAT	Вещественное значение float

Возвращают формат хранимых в изображении данных.

int get_image_channel_order (image1d_t image)

int get_image_channel_order (image2d_t image)

int get_image_channel_order (image3d_t image)

int get_image_channel_order (image1d_array_t image)

int get_image_channel_order (image2d_array_t image)

Возвращаемое значение является одной из следующих констант (таблица 4).

Таблица 4. Константы форматов данных изображения

Константа формата	Содержание ячеек изображения
CL_R, CL_Rx	(r, 0.0, 0.0, 1.0)
CL_A	(0.0, 0.0, 0.0, a)
CL_RG, CL_RGx	(r, g, 0.0, 1.0)
CL_RA	(r, 0.0, 0.0, a)
CL_RGB, CL_RGBx	(r, g, b, 1.0)
CL_RGBA, CL_BGRA, CL_ARGB	(r, g, b, a)
CL_INTENSITY	(I, I, I, I)
CL_LUMINANCE	(L, L, L, 1.0)

Таблица 5. Соответствие константы формата константе типа изображения

Константы формата	Применимые константы типа
CL_R, CL_Rx, CL_A, CL_RG, CL_RGx, CL_RA, CL_RGBA	Любые
CL_INTENSITY, CL_LUMINANCE	CL_UNORM_INT8, CL_UNORM_INT16, CL_SNORM_INT8, CL_SNORM_INT16, CL_HALF_FLOAT, CL_FLOAT
CL_RGB, CL_RGBx	CL_UNORM_SHORT_565, CL_UNORM_SHORT_555, CL_UNORM_INT101010
CL_BGRA, CL_ARGB	CL_UNORM_INT8, CL_SNORM_INT8, CL_SIGNED_INT8, CL_UNSIGNED_INT8

Константа CL_UNORM_SHORT_565 предполагает, что цвет записывается через: Red(Красный) - 5 бит, Green(Зеленый) - 6 бит, Blue(Синий) - 5 бит. Константы CL_UNORM_SHORT_555 и CL_UNORM_INT101010 имеют аналогичный смысл.

Особенности работы с изображениями

__read_only, __write_only, __read_write - спецификаторы для объявления любых переменных (включая и изображения), описывающие права доступа к переменной (только для чтения, только для записи, четные и запись). В коде программы задаются например так "**__read_only image2d_t MyImage**".

В ранних версиях OpenCL невозможно проводить операции чтения и записи для одного и того же изображения, даже при указании **__read_write**.

Ниже рассмотрим функции создания изображений и чтения/записи в изображения.

cl_mem clCreateImage (cl_context context, cl_mem_flags flags, const cl_image_format *image_format, const cl_image_desc *image_desc, void *host_ptr, cl_int *errcode_ret)

Функция создает 1,2,3-мерное изображение или массив 1,2-мерных изображений в контексте context.

flags - свойства объекта памяти (полностью аналогичны одноименному параметру функции clCreateBuffer).

image_format - указатель на структуру формата изображения cl_image_format, совмещающую константу типа (см. выше табл. 3) и константу формата (см. выше табл. 4).

```
struct cl_image_format
{ cl_channel_order image_channel_order; //константа типа
```

```
cl_channel_type image_channel_data_type; } //константа формата
```

image_desc - указатель на структуру cl_image_desc, описывающую тип и размеры изображения.

```
struct cl_image_desc
{
    cl_mem_object_type image_type; /* тип изображения:
CL_MEM_OBJECT_IMAGE1D,      CL_MEM_OBJECT_IMAGE1D_ARRAY,
CL_MEM_OBJECT_IMAGE2D,      CL_MEM_OBJECT_IMAGE2D_ARRAY,
CL_MEM_OBJECT_IMAGE3D */
    size_t image_width; //ширина (размер по x)
    size_t image_height; //высота (размер по y)
    size_t image_depth; //глубина (размер по z)
    size_t image_array_size; //количество изображений в массиве
    //ниженеречисленные параметры зануляются при host_ptr=NULL
    size_t image_row_pitch; size_t image_slice_pitch;
    cl_uint num_mip_levels; cl_uint num_samples; cl_mem buffer; }
```

host_ptr - адрес ранее выделенного участка памяти, если назначить его как NULL, то данная опция не используется.

errcode_ret - по этому адресу функция записывает код ошибки или CL_SUCCESS в случае успешного выполнения.

cl_mem clCreateImage2D (cl_context context, cl_mem_flags flags, const cl_image_format *image_format, size_t image_width, size_t image_height, size_t image_row_pitch, void *host_ptr, cl_int *errcode_ret)

Функция создает 2-мерное изображение в контексте context со свойствами объекта памяти flags и форматом image_format, шириной image_width, высотой image_height. В errcode_ret функция записывает код ошибки или CL_SUCCESS в случае успешного выполнения. host_ptr - указатель на ранее созданное изображение, image_row_pitch зануляется при host_ptr=NULL, в противном случае = image_width * размер элемента изображения в байтах.

cl_mem clCreateImage3D (cl_context context, cl_mem_flags flags, const cl_image_format *image_format, size_t image_width, size_t image_height, size_t image_depth, size_t image_row_pitch, size_t image_slice_pitch, void *host_ptr, cl_int *errcode_ret)

Функция создает 3-мерное изображение в контексте context со свойствами объекта памяти flags и форматом image_format, шириной image_width, высотой image_height, глубиной image_depth. В errcode_ret функция записывает код ошибки или CL_SUCCESS в случае успешного

выполнения. *host_ptr* - указатель на ранее созданное изображение, *image_row_pitch* зануляется при *host_ptr=NULL*, в противном случае = *image_width* * размер элемента изображения в байтах. *image_slice_pitch* зануляется при *host_ptr=NULL*, в противном случае = *image_height* * размер элемента изображения в байтах.

Функции *clCreateFromGLTexture*, *clCreateFromGLTexture2D*, *clCreateFromGLRenderbuffer* создают изображения из текстур OpenGL. Также есть ряд функций, создающих изображения на основе текстур Direct3D, это *clCreateFromD3D10Texture2DKHR* и подобные. Функций, создающих изображения по указанному файлу графического формата на диске, (по крайней мере, в ранних версиях OpenCL) нет.

cl_int clEnqueueWriteImage (cl_command_queue command_queue, cl_mem image, cl_bool blocking_write, const size_t *origin, const size_t *region, size_t input_row_pitch, size_t input_slice_pitch, const void * ptr, cl_uint num_events_in_wait_list, const cl_event *event_wait_list, cl_event *event)

Копирование данных в изображение.

command_queue - очередь команд.

image - изображение или массив изображений.

blocking_write - если равен CL_TRUE, то ожидать завершения копирования, если CL_FALSE, то не ожидать.

origin - смещение координат для копирования данных в изображение, задается в виде {x, 0, 0}, {x, y, 0} или {x, y, z}.

region - (ширина, высота, глубина) записываемого участка в пикселях, задается в виде {x, 1, 1}, {x, y, 1} или {x, y, z}.

input_row_pitch - размер каждой строки в байтах, при указании нуля автоматически рассчитывается как размер каждого элемента в байтах * ширина.

input_slice_pitch - размер глубины строки в байтах, для 1,2-мерных изображений этот параметр зануляется, при указании нуля автоматически рассчитывается как *input_row_pitch**высота.

ptr - указатель на область памяти, которая копируется в изображение.

event_wait_list, *num_events_in_wait_list*, *event* служат для более детального управления очередью команд, при неиспользовании зануляются.

Функция возвращает CL_SUCCESS в случае успешного выполнения.

cl_int clEnqueueFillImage (cl_command_queue command_queue,

cl_mem image, const void *fill_color, const size_t *origin, const size_t *region, cl_uint num_events_in_wait_list, const cl_event *event_wait_list, cl_event *event)

Функция заполняет все изображение или его часть одним цветом *fill_color*, где цвет задается четверкой параметров {r, g, b, a} и должен соотноситься с форматом/типом изображения.

cl_int clEnqueueReadImage (cl_command_queue command_queue, cl_mem image, cl_bool blocking_read, const size_t *origin, const size_t *region, size_t row_pitch, size_t slice_pitch, void *ptr, cl_uint num_events_in_wait_list, const cl_event *event_wait_list, cl_event *event)

Чтение данных из изображения в область памяти.

command_queue - очередь команд.

image - изображение или массив изображений.

blocking_read - если равен CL_TRUE, то ожидать завершения чтения, если CL_FALSE, то не ожидать.

origin - смещение координат для чтения данных из изображения, задается в виде {x, 0, 0}, {x, y, 0} или {x, y, z}.

region - (ширина, высота, глубина) считываемого участка в пикселях, задается в виде {x, 1, 1}, {x, y, 1} или {x, y, z}.

input_row_pitch - размер каждой строки в байтах, при указании нуля автоматически рассчитывается как размер каждого элемента в байтах * ширина.

input_slice_pitch - размер глубины строки в байтах, для 1,2-мерных изображений этот параметр зануляется, при указании нуля автоматически рассчитывается как *input_row_pitch**высота.

ptr - указатель на область памяти, в которую копируются данные из изображения.

event_wait_list, *num_events_in_wait_list*, *event* служат для более детального управления очередью команд, при неиспользовании зануляются.

Функция возвращает CL_SUCCESS в случае успешного выполнения.

Пример использования изображения в OpenCL программе

Опишем функцию-ядро, получающую изображение только для чтения и изображение только для записи. В ней каждый рабочий элемент будет обрабатывать соответствующую его номеру строку изображения, он будет записывать во второе изображение цвет первого, умноженный на два. Нулевой рабочий элемент, просто для иллюстрации, будет выводить на эк-

ран цвет изображения из ячейки [0][0].

```
const char * opencl_source =
"__kernel void Kernel(__read_only image2d_t imageRead,  \n\"
"                __write_only image2d_t imageWrite) \n\"
"{int i=get_global_id(0), j; int2 coord; coord.s0=i;      \n\"
"int M=get_image_height(imageRead);                      \n\"
"for(j=0;j<M;j++) { coord.s1=j;                          \n\"
"write_imagef (imageWrite, coord, read_imagef(imageRead, coord)*2) ;
}\n\"
"if (i==0) { coord.s1=0 ; float4 C=read_imagef(imageRead, coord) ; \n\"
"printf(\"\\nColor [0][0] = %f, %f, %f, %f.\\n\",C.s0,C.s1,C.s2,C.s3);} }\n\";
```

Зададим код OpenCL программы, выполняющийся на процессоре.

```
//определение динамического двумерного массива
int N, M ;
printf("\nВведите число строк и число столбцов двумерного массива
");
scanf("%d",&N) ; scanf("%d",&M) ;
if (N<2 || M<2) {printf("\nN и M должны быть >= 2!") ; return -1 ; }
float **A ; A=new float *[N] ;
for (int i=0 ; i<N ; i++) A[i]=new float [M] ;
//ввод элементов массива
for (int i=0 ; i<N ; i++) for (int j=0 ; j<M ; j++)
{ printf("Введите A[%d][%d] ",i,j) ; scanf("%f",&A[i][j]) ; }

//описание формата изображения
cl_image_format image_format ;
image_format.image_channel_data_type = CL_FLOAT ;
image_format.image_channel_order = CL_A ;
//здесь тип CL_FLOAT соотносится с типом массива float **A ;
//безопасные форматы данных для "имитации массива
изображением"
//это CL_A и CL_R, для float также CL_INTENSITY и
CL_LUMINANCE

//выделяем память под изображения на OpenCL устройстве
cl_mem imageRead = clCreateImage2D (context,
CL_MEM_READ_ONLY, &image_format, N, M, 0, 0, 0) ;
```

```

    cl_mem    imageWrite    =    clCreateImage2D    (context,
CL_MEM_WRITE_ONLY, &image_format, N, M, 0, 0, 0) ;

    //копируем в imageRead построчно динамический двумерный массив
A
    size_t origin[3]={0,0,0}; //изначально копируем начиная с нулевого
индекса
    size_t region[3]={1,M,1}; //регион под одну строку длиной M
    for (int i=0 ; i<N ; i++)
    {
        err=clEnqueueWriteImage    (command_queue,    imageRead,
CL_TRUE, origin, region, 0, 0, (void *)A[i], 0, 0, 0) ;
        if ( err    !=    CL_SUCCESS    ) {    printf("\nОшибка
clEnqueueWriteImage, Номер строки=%d",i) ; return -1 ; }
        //следующую строку нужно копировать с нового индекса
(смещения)
        origin[0]++ ;
    }

    //задание нулевого аргумента ядра
    err    =    clSetKernelArg    (kernel,    0,    sizeof(imageRead),    (void
*)&imageRead) ;
    if (err!=CL_SUCCESS) { printf("\nОшибка вызова clSetKernelArg
imageRead"); return -1 ; }
    //задание первого аргумента ядра
    err    =    clSetKernelArg    (kernel,    1,    sizeof(imageWrite),    (void
*)&imageWrite) ;
    if (err!=CL_SUCCESS) { printf("\nОшибка вызова clSetKernelArg
imageWrite"); return -1 ; }

    //ЗАПУСК ЯДРА
    size_t sizeAll=N; //рабочих элементов столько, сколько строк
    err    =    clEnqueueNDRangeKernel    (command_queue, kernel, 1, NULL,
&sizeAll, NULL, 0, 0, 0) ;
    if (err!=CL_SUCCESS) { printf("\nОшибка запуска ядра"); return -1 ; }

    //ждемся завершения выполнения ядра всеми элементами
    err    =    clFinish(command_queue) ;

```

```

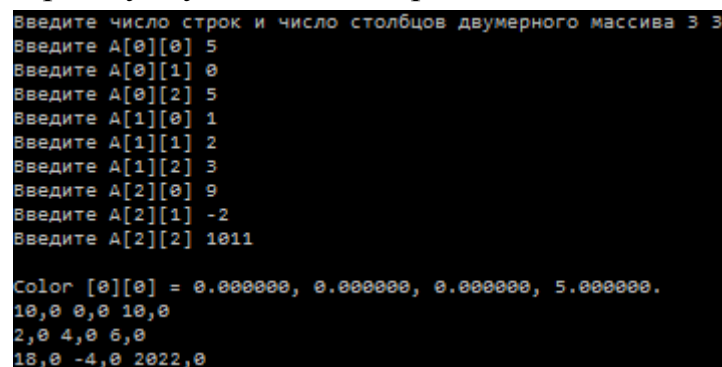
    if (err!=CL_SUCCESS) { printf("\nОшибка завершения ядра"); return -
1 ; }

    //получение результатов вычислений ядра
    //считываем данные из изображения в массив
    origin[0]=0 ; origin[1]=0 ; origin[2]=0 ;
    for (int i=0 ; i<N ; i++)
    {
        err=clEnqueueReadImage (command_queue, imageWrite,
CL_TRUE, origin, region, 0, 0, (void *)A[i], 0, 0, 0) ;
        if ( err != CL_SUCCESS ) { printf("\nОшибка
clEnqueueReadImage, i=%d",i) ; return -1 ; }
        //следующую строку нужно считывать с нового индекса
(смещения)
        origin[0]++ ;
    }

    //вывод результатов на экран
    for (int i=0 ; i<N ; i++) { printf("\n") ;
    for (int j=0 ; j<M ; j++) printf("%.1f ",A[i][j]) ; }

```

Запустим программу и убедимся в ее работоспособности (рис.14).



```

Введите число строк и число столбцов двумерного массива 3 3
Введите A[0][0] 5
Введите A[0][1] 0
Введите A[0][2] 5
Введите A[1][0] 1
Введите A[1][1] 2
Введите A[1][2] 3
Введите A[2][0] 9
Введите A[2][1] -2
Введите A[2][2] 1011

Color [0][0] = 0.000000, 0.000000, 0.000000, 5.000000.
10,0 0,0 10,0
2,0 4,0 6,0
18,0 -4,0 2022,0

```

Рис. 14. Выполнение OpenCL программы, использующей изображения

В примере изображение использовано так, что каждая ячейка изображения хранит одно число. А можно применять и схему CL_RGBA (совместимую с любой константой типа), при которой одна ячейка изображения хранит четыре значения. В этом случае в примере выше двумерный массив надо объявить не как $[N][M]$, а как $[N][M*4]$, при этом копирование/чтение в изображение будет осуществляться тем же регионом $\{1,M,1\}$. Распределение элементов массива в ячейки изображения произойдет так (рис. 15).

$$N = 3, M = 2, M * 4 = 8$$

Массив 3×8

1	2	3	4	5	6	7	8
0	0	0	0	2	2	2	2
4	3	2	1	9	8	9	8

Изображение 3×2

(1 2 3 4)	(5 6 7 8)
(0 0 0 0)	(2 2 2 2)
(4 3 2 1)	(9 8 9 8)

Рис. 15. Распределение элементов массива в ячейки изображения (CL_RGBA)

Порядок выполнения лабораторной работы

1) Разработайте OpenCL программу, решающую задачу из вашего варианта и **НЕ использующую изображения**.

2) Протестируйте работу OpenCL программы и убедитесь, что она выполняется корректно.

3) Разработайте OpenCL программу, решающую задачу из вашего варианта с **использованием изображений**.

4) Протестируйте работу OpenCL программы и убедитесь, что она выполняется корректно.

Варианты заданий

1) Пользователь вводит размерность квадратного двумерного массива. Заполнить массив по следующему правилу.

0,1 0,2 0,3 0,4

1,2 1,3 1,4 0,5

1,1 1,6 1,5 0,6

1,0 0,9 0,8 0,7

2) Пользователь вводит размерность квадратного двумерного массива. Заполнить массив по следующему правилу.

0 1 2 3

-1 0 1 2

-2 -1 0 1

-3 -2 -1 0

Вычислить средние арифметические каждой строки.

3) Пользователь вводит число строк и столбцов двумерного массива, а также пару чисел a , b и натуральное число K . Заполнить двумерный массив "шахматными клетками" цвета a , b , длиной стороны K .

a a b b a a b

a a b b a a b

b b a a b b a

b b a a b b a (Здесь $K = 2$)

4) Из файла считывается число строк и столбцов двумерного массива и его элементы. Вычислить суммы элементов каждой строки массива.

5) В файле заданы две матрицы. Вычислить их произведение (по математическому правилу, а не перемножив $a[i][j]$ на $b[i][j]$).

6) Из файла считывается число строк и столбцов двумерного массива и его элементы, равные 0 или 1. Обвести все формы, состоящие из единиц, контуром из двоек.

	0 0 0 0 1		2 2 2 2 1
	0 1 1 0 0		2 1 1 2 2
	0 1 0 0 0		2 1 2 2 0
Было	0 0 0 0 0	Стало	2 2 2 0 0

7) В файле задана матрица 4x4. Вычислить минор каждого элемента.

8) Из файла считывается число строк и столбцов двумерного массива и его элементы. Вычислить произведения элементов каждого столбца массива.

9) Из файла считывается число строк и столбцов двумерного массива, его элементы и число K . Вычислить число вхождений K в массив и число вхождений K среди элементов, стоящих на главной диагонали.

10) Пользователь вводит число строк и столбцов двумерного массива. Заполнить массив по следующему правилу.

0 1 2 3 4 5	
1 2 3 4 5 6	Вычислить суммы каждого столбца.
2 3 4 5 6 7	

11) Пользователь вводит размерность квадратного двумерного массива и числа a, b, c, d, e . Заполнить массив по следующему правилу.

a b b b a
c a b a d
c c a d d
c a e a d
a e e e a

12) Из файла считывается число строк и столбцов двумерного массива A и его элементы, также число строк и столбцов двумерного массива B и его элементы. Найти число вхождений массива B в A .

13) Из файла считывается число строк и столбцов двумерного массива и его элементы, а также количество пар чисел (a_i, b_i) и их значения. Заменить в массиве каждый a_i на b_i .

14) Пользователь вводит число строк и столбцов двумерного массива, а также номер строки str , номер столбца stl и числа a, b, c, d, e, f . Заполнить массив по следующему правилу.

c c **b** d d

a a a a a

e e **b** f f

e e **b** f f (Здесь str=1, stl=2)

15) Пользователь вводит число строк и столбцов двумерного массива, а также числа a, b. Заполнить массив по следующему правилу.

a b a b a b

b a b a b a

a b a b a b

16) Из файла считывается число строк и столбцов двумерного массива и его элементы, а также числа a, b. Заменить каждый элемент, среднее арифметическое окружающих элементов которого равно a, на b.

17) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти номер строки, содержащей наиболее длинную последовательность подряд идущих одинаковых значений.

18) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти в массиве максимальный по площади прямоугольник, состоящий из одинаковых значений, и его координаты.

19) Из файла считывается размерность квадратного двумерного массива и его элементы. Определить является ли массив магическим квадратом. Для магического квадрата суммы элементов каждой диагонали, параллельной главной, равны между собой.

20) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти наиболее длинную X-образную последовательность одинаковых элементов.

0 1 0 1

0 0 1 0

5 1 0 1

21) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти максимальные элементы в каждой строке массива.

22) Из файла считывается число строк и столбцов двумерного массива и его элементы, а также число K. Найти количество содержащихся в заданном массиве квадратных подмассивов размерности 2x2, сумма элементов которых равна K.

23) Из файла считывается размерность квадратного двумерного массива и его элементы. Найти все индексы i, для которых i-я строка и i-й

столбец содержат одинаковое количество четных чисел.

24) Из файла считывается число строк и столбцов двумерного массива и его элементы, а также цифра K . Найти номер строки, содержащей наибольшее количество чисел, включающих в свою запись цифру K .

25) Из файла считывается число строк и столбцов двумерного массива и его элементы. Для каждой строки определить сколько ее элементов являются делителями для нулевого элемента этой строки.

26) Из файла считывается размерность N одномерного массива A и его элементы. Составить двумерный массив $[N][9]$, записав в него остатки от деления каждого $A[i]$ на $2, 3, 4, \dots, 10$.

27) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти номера тех столбцов, элементы которых образуют произвольную арифметическую прогрессию.

28) Из файла считывается число строк и столбцов двумерного массива и его элементы, а также число K . Заменить все элементы строки: 1) начинающейся и заканчивающейся числом K , нулями; 2) начинающейся числом K , единицами; 3) заканчивающейся числом K , двойками.

29) Из файла считывается число строк и столбцов двумерного массива A и его элементы, также размерность одномерного массива B и его элементы. Найти число вхождений массива B в A , учитывая размещение массива B только по строкам массива A .

30) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти номер столбца, содержащего наиболее длинную последовательность подряд идущих значений вида $\{n, n+1, n+2, \dots, n+k\}$.

31) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти индексы строк, содержащих хотя бы одно простое число.

32) Из файла считывается число строк и столбцов двумерного массива и его элементы. Для каждого столбца найти наибольший общий делитель его элементов.

33) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти определители каждого входящего в массив подмассива размерности 2×2 .

34) Из файла считывается размерность квадратного двумерного массива и его элементы. Составить новый массив на основе считанного, повысив в новом массиве: 1) элементы побочной диагонали на 1; 2) эле-

менты, стоящие выше побочной диагонали, на 2; 3) элементы, стоящие ниже побочной диагонали, на 3.

35) Из файла считывается число строк и столбцов двумерного массива и его элементы. Отсортировать каждый столбец массива по убыванию.

36) Из файла считывается число строк и столбцов двумерного массива и его элементы. Переставить в массиве строки в порядке возрастания сумм их элементов.

37) Из файла считывается нечетная размерность квадратного двумерного массива и его элементы. Переставить строки и столбцы в массиве таким образом, чтобы максимальный элемент располагался в середине массива (на пересечении главной и побочной диагоналей).

38) Пользователь вводит натуральную степень n . Записать в массив треугольник паскаля от 1 степени до n .

39) Пользователь вводит число строк и столбцов двумерного массива. Заполнить массив по следующему правилу.

```
1 2 3 4 5 6
12 11 10 9 8 7
13 14 15 16 17 18
24 23 22 21 20 19
```

40) Пользователь вводит число строк и столбцов двумерного массива. Заполнить массив по следующему правилу.

```
1 1 1 1
1 2 2 2
1 2 3 3
1 2 3 4
```

41) Из файла считывается размерность квадратного двумерного массива и его элементы. Массив описывает шахматное поле с произвольным количеством ферзей. Для каждого ферзя определить, существует ли угрожающий ему ферзь.

42) Из файла считывается число строк и столбцов двумерных массивов, элементы массива A и элементы массива B . Составить новый массив C , занося в него элементы из A , если соответствующие им элементы из B отличны от нуля, в противном случае в массив C заносится ноль.

43) Из файла считывается число точек и их координаты в трехмерном пространстве. Найти точку, максимально удаленную от начала координат.

44) Из файла считывается размерность одномерного массива, его

элементы и натуральное число K . Составить двумерный массив, записывая в его i -ую строку результаты возведения i -ого элемента в степени $1, 2, 3, \dots, K$.

45) Из файла считывается количество дат и сами даты в формате год, месяц, день, час, минута, секунда. Найти максимальную и минимальную даты, не учитывая при этом некорректные даты (допускается упрощенная проверка дня как значения из интервала $[1, 31]$).

46) Пользователь вводит четное число строк и столбцов двумерного массива. Заполнить массив по следующему правилу.

```
1 2 5 6
3 4 7 8
9 10 13 14
11 12 15 16
```

47) Из файла считывается размерность квадратного двумерного массива и его элементы. Найти номер диагонали, параллельной побочной, имеющей минимальное среднее арифметическое входящих в нее элементов.

48) Из файла считывается размерность одномерного массива и его элементы. Составить двумерный массив, записывая в его i -ую строку наибольший общий делитель i -ого элемента с каждым другим элементом.

49) Из файла считывается число точек и их координаты в двумерном пространстве. Составить двумерный массив, записывая в его i -ую строку, расстояния между i -ой точкой и всем другими точками.

50) Пользователь вводит число строк и столбцов двумерного массива. Заполнить массив по следующему правилу.

```
1 0 2 0 3
0 4 0 5 0
6 0 7 0 8
```

51) Из файла считывается число строк и столбцов двумерного массива и его элементы. Переставить строки (если это возможно) таким образом, чтобы максимальный элемент располагался в последней строке, а минимальный в первой.

52) Из файла считывается число строк и столбцов двумерного массива и его элементы. Переставить в массиве столбцы в порядке убывания их максимальных элементов.

53) Из файла считывается число строк и столбцов двумерного массива и его элементы. Отсортировать каждую строку массива по возрастанию.

54) Из файла считывается размерность квадратного двумерного массива и его элементы. Найти определители каждого входящего в массив подмассива размерности 3×3 .

55) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти номера тех строк, элементы которых образуют произвольную геометрическую прогрессию.

56) Из файла считывается число строк и столбцов двумерного массива и его элементы. Для каждой строки найти все общие делители её элементов.

57) Из файла считывается размерность квадратного двумерного массива и его элементы. Найти номер диагонали, параллельной главной, содержащей наиболее длинную последовательность подряд идущих одинаковых значений.

58) Из файла считывается число строк и столбцов двумерного массива A и его элементы, также размерность одномерного массива B и его элементы. Найти число вхождений массива B в A , учитывая размещение массива B только по столбцам массива A .

59) Из файла считывается размерность квадратного двумерного массива и его элементы. Определить является ли массив магическим квадратом. Для магического квадрата суммы элементов каждой строки и каждого столбца одинаковые.

60) Из файла считывается число строк и столбцов двумерного массива и его элементы. Найти минимальные элементы в каждом столбце массива.

Лабораторная работа №5

Тема: События. Динамическое составление OpenCL программы

Цели: Ознакомиться с возможностью динамического составления OpenCL программ, тем самым, получить навык адаптации кода OpenCL программ к требованиям пользователя. Изучить механизм событий, получить навык управления событиями в OpenCL программе.

События

Событие может быть определено с помощью двух типов: `cl_event` - событие в OpenCL программе, выполняющейся на CPU, `event_t` - событие в OpenCL программе, выполняющейся на GPU.

Событие `event_t` связано с операциям копирования данных между локальной памятью рабочей группы и глобальной памятью устройства и (по крайней мере в ранних версиях OpenCL) не может быть переопределено программистом для обозначения выполнения произвольных операций.

`event_t async_work_group_copy (__local тип *dst, const __global тип *src, size_t num_gentypes, event_t event)`

`event_t async_work_group_copy (__global тип *dst, const __local тип *src, size_t num_gentypes, event_t event)`

Асинхронное копирование `num_gentypes` элементов из `src` в `dst`, т.е. из локальной памяти в глобальную или наоборот. Функция должна вызываться всеми рабочими элементами из одной группы. Функция возвращает событие `event_t`, обозначающее окончание копирования данных. Аргумент `event` может указываться нулевым, в противном случае это событие предыдущего асинхронного копирования в той же рабочей группе, с которым произойдет синхронизация.

`event_t async_work_group_strided_copy (__local тип *dst, const __global тип *src, size_t num_gentypes, size_t stride, event_t event)`

`event_t async_work_group_strided_copy (__global тип *dst, const __local тип *src, size_t num_gentypes, size_t stride, event_t event)`

Функции аналогичные предыдущим, кроме того, что осуществляют копирование с шагом `stride`. Элементы из `src` берутся последовательно, а в `dst` заносятся с указанным шагом.

`void wait_group_events (int num_events, event_t *event_list)`

Ожидание завершения массива событий `event_list` размерности `num_events` элементов. Функция должна вызываться всеми рабочими

элементами из одной группы.

Функция-ядро, использующая события `event_t` может быть, например, такой.

```
//функция получает массив A и его размерность N
//предполагаем, что групп может быть несколько, но каждая из 32
элементов
//и для числа раб. элементов и для числа групп используется только 1
измерение
__kernel void Kernel(__global int *A, int N) \n"
//idL-локальный ИД элемента (от 0 до 31), idG-номер группы (от 0
до ?)
"{ int idL=get_local_id(0), idG = get_group_id(0) ; \n"
//каждая группа создает массив на 32 элемента
__local int B[32] ; \n"
//копирование 32 элементов из участка массива A в массив B (рис.
16)
"event_t event = async_work_group_copy(B, A+32*idG, 32, 0); \n"
//ждем когда копирование завершится
"wait_group_events (1, &event) ; \n"
//раб. элементы пересчитывают один элемент из массива B каждый
"B[idL] = F(B[idL]) ; \n"
//копирование 32 элементов из массива B в участок массива A (рис.
16)
"async_work_group_copy(A+32*idG,B,32,0); } \n"
```

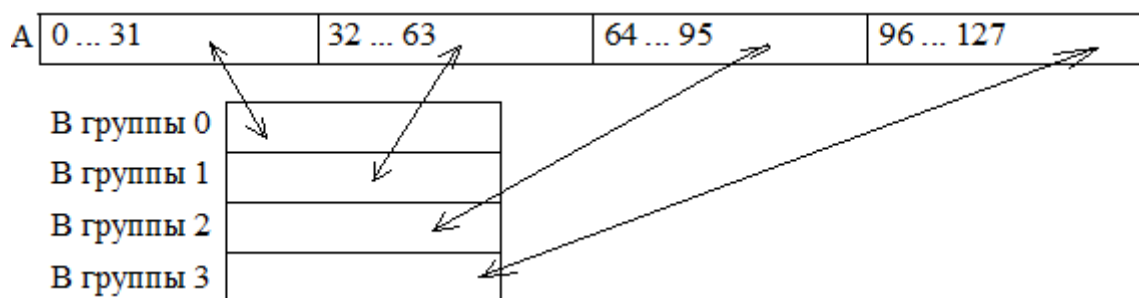


Рис. 16. Копирование между глобальной (A) и локальной (B) памятью

События `cl_event` встречались в тексте ранее, например, в таких функциях как чтение/запись в буфер или изображение. Подобные функции обладали тройкой следующих параметров.

```
( cl_uint num_events_in_wait_list, const cl_event *event_wait_list,
cl_event *event )
```

event_wait_list - массив событий, которые должны быть выполнены до запуска операций функции (контекст и очередь команд этих событий должны совпадать с контекстом и очередью команд текущей функции).

num_events_in_wait_list - количество событий в массиве event_wait_list.

event - автоматически генерируемое функцией событие, обозначающее ее выполнение, это событие можно использовать в других массивах событий, которые должны быть предварительно выполнены до запуска некоторого другого действия. Неправильно указывать в качестве адреса *event используемый адрес из *event_wait_list (т.е. ожидать собственного выполнения до собственного запуска).

Другие функции для работы с событиями cl_event.

cl_int clGetEventInfo (cl_event event, cl_event_info param_name, size_t param_value_size, void *param_value, size_t *param_value_size_ret)

Функция возвращает информацию об указанном событии event. Параметр param_name является специальной константой, описывающей вид запрашиваемой информации.

Функция заносит запрашиваемую информацию по адресу *param_value, а в *param_value_size_ret записывает реальный объем данных, занесенных в *param_value. Через параметр param_value_size разработчик указывает ожидаемый размер данных, которые будут помещены в *param_value.

Константы для param_name.

CL_EVENT_COMMAND_QUEUE - очередь команд события (тип cl_command_queue).

CL_EVENT_CONTEXT - контекст события (тип cl_context).

CL_EVENT_COMMAND_TYPE - команда, с которой связано событие (тип cl_command_type), может принимать одно из следующих значений: CL_COMMAND_NDRANGE_KERNEL, CL_COMMAND_TASK, CL_COMMAND_NATIVE_KERNEL, CL_COMMAND_READ_BUFFER, CL_COMMAND_WRITE_BUFFER, CL_COMMAND_COPY_BUFFER, CL_COMMAND_READ_IMAGE, CL_COMMAND_WRITE_IMAGE, CL_COMMAND_COPY_IMAGE, CL_COMMAND_COPY_BUFFER_TO_IMAGE, CL_COMMAND_COPY_IMAGE_TO_BUFFER, CL_COMMAND_MAP_BUFFER, CL_COMMAND_MAP_IMAGE, CL_COMMAND_UNMAP_MEM_OBJECT, CL_COMMAND_MARKER,

CL_COMMAND_ACQUIRE_GL_OBJECTS,
CL_COMMAND_RELEASE_GL_OBJECTS,
CL_COMMAND_READ_BUFFER_RECT,
CL_COMMAND_WRITE_BUFFER_RECT,
CL_COMMAND_COPY_BUFFER_RECT, CL_COMMAND_USER,
CL_COMMAND_BARRIER, CL_COMMAND_MIGRATE_MEM_OBJECTS,
CL_COMMAND_FILL_BUFFER, CL_COMMAND_FILL_IMAGE,
CL_COMMAND_GL_FENCE_SYNC_OBJECT_KHR.

CL_EVENT_COMMAND_EXECUTION_STATUS - статус выполнения команды, с которой связано событие (тип cl_int), может принимать одно из следующих значений:

- CL_QUEUED - команда ассоциирована с очередью команд,
- CL_SUBMITTED - команда направлена к устройству,
- CL_RUNNING - команда в процессе выполнения,
- CL_COMPLETE - команда выполнена,
- отрицательное число - код ошибки.

**cl_int clSetEventCallback (cl_event event,
cl_int command_exec_callback_type,
void (CL_CALLBACK *pfn_event_notify) (cl_event event, cl_int
event_command_exec_status, void *user_data), void *user_data)**

На достижение состояния command_exec_callback_type события event функция назначает функцию обратного вызова pfn_event_notify с аргументами (cl_event event, cl_int event_command_exec_status, void *user_data).

Последний аргумент user_data определяет данные, которые поступят в одноименный аргумент функции обратного вызова pfn_event_notify.

Состояние события command_exec_callback_type это CL_SUBMITTED, CL_RUNNING или CL_COMPLETE, при этом CL_COMPLETE здесь означает как выполнение связанной с событием команды, так и ее аварийное прекращение.

cl_event clCreateUserEvent (cl_context context, cl_int *errcode_ret)

Функция создает и возвращает пользовательское событие, связанное с контекстом context. В errcode_ret функция записывает код ошибки или CL_SUCCESS если ошибки не возникло. Пользовательское событие создается с состоянием CL_SUBMITTED.

cl_int clSetUserEventStatus (cl_event event, cl_int execution_status)

Функция устанавливает состояние execution_status пользовательского

события event, execution_status может быть равен CL_COMPLETE или отрицательному числу. Перевод состояния пользовательского события назад в CL_SUBMITTED эффекта не даст.

cl_int clWaitForEvents (cl_uint num_events, const cl_event *event_list)

Ожидание завершения массива событий event_list размерности num_events. Функция возвращает CL_SUCCESS если все события выполнились со статусом CL_COMPLETE. Возвращает CL_EXEC_STATUS_ERROR_FOR_EVENTS_IN_WAIT_LIST если какие-то события завершились со статусом ошибки (отр. числом) или возвращает другой код ошибки.

cl_int clEnqueueMarkerWithWaitList (cl_command_queue command_queue, cl_uint num_events_in_wait_list, const cl_event *event_wait_list, cl_event *event)

Ожидание завершения событий из массива event_wait_list размерности num_events_in_wait_list. Функция создает событие event, связанное с ее выполнением.

Применение событий cl_event может быть, например, таким.

Пример 1. Функция записи в буфер (clEnqueueWriteBuffer) дожидается завершения двух операций чтения из буфера, запущенных процессором в режиме "не ожидать завершения" (CL_FALSE).

```
cl_event event[2];
clEnqueueReadBuffer (command_queue, mas_for_GPU_A, CL_FALSE,
0, N*sizeof(int), (void *)A, 0, 0, &event[0]);
clEnqueueReadBuffer (command_queue, mas_for_GPU_B, CL_FALSE,
0, N*sizeof(int), (void *)B, 0, 0, &event[1]);
clEnqueueWriteBuffer (command_queue, mas_for_GPU_C, CL_TRUE,
0, N*sizeof(int), (const void *)C, 2, event, 0);
```

Пример 2. Пользовательское событие связываем с окончанием ввода размерности массива пользователем.

```
//функция обратного вызова для события
void __stdcall func_cl_event (cl_event event, cl_int
event_command_exec_status, void *user_data)
{ printf("\n\tПользовательское событие ");
if (event_command_exec_status==CL_COMPLETE)
printf("успешно выполнено"); }
```

```

//в функции main создаем пользовательское событие
cl_event my_event ;
my_event = clCreateUserEvent (context, &arg_int) ;
if (arg_int!=CL_SUCCESS){printf("\nОшибка создания события");
return -1;}
//назначаем ему функцию обратного вызова
arg_int      =      clSetEventCallback(my_event,      CL_COMPLETE,
func_cl_event, NULL) ;
if (arg_int!=CL_SUCCESS) { printf("\nОшибка назначения функции
события") ; return -1 ; }
//вводим размерность массива
int N=0 ;
while (N<2) { printf("\nВведите число элементов массива (>1): ") ;

scanf("%d",&N) ; }
//меняем статус пользовательского события на "выполнено"
(CL_COMPLETE)
arg_int = clSetUserEventStatus (my_event, CL_COMPLETE) ;
if (arg_int!=CL_SUCCESS) { printf("\nОшибка clSetUserEventStatus") ;
return -1 ; }
//ожидаем завершения события
arg_int = clWaitForEvents (1, &my_event) ;
if (arg_int!=CL_SUCCESS) {printf("\nОшибка clWaitForEvents");
return -1 ; }

```

Динамическое составление OpenCL программы

Код OpenCL программы, выполняющейся на GPU, задается в виде текста, массивом строк или одной строкой. Эта особенность позволяет динамически менять/вставлять/вырезать этот текст исходя из условий решаемой задачи и входных данных пользователя.

Рассмотрим конкретный пример.

Пусть задан код функции-ядра, рассчитанной на работу только одной группы с произвольным числом раб. элементов.

```

const char * opencl_source =
"__kernel void Kernel(__global int *A, int N) \n"
"{ int i, id=get_global_id(0), kol=get_global_size(0) ; \n"
"__local int rez ; rez=0 ; barrier(CLK_LOCAL_MEM_FENCE) ; \n"
"for (i=id+1 ; i<N ; i=i+kol) atomic_add(&rez,A[i]); \n"

```

```
"if (id==0) A[0]=rez ; }\n" ;
```

Функция производит суммирование всех элементов массива A с индексами от 1 до N-1 и сохраняет найденную сумму в нулевой ячейке этого же массива.

Изменим текст функции-ядра.

```
const char * opencl_source =  
"__kernel void Kernel(__global int *A, int N) \n"\n"{ int i, id=get_global_id(0), kol=get_global_size(0) ; \n"\n"__local int rez ; rez=@ ; barrier(CLK_LOCAL_MEM_FENCE) ; \n"\n"for (i=id+1 ; i<N ; i=i+kol) ?(&rez,A[i]); \n"\n"if (id==0) A[0]=rez ; }\n" ;
```

При попытке компиляции такого кода естественно произойдет ошибка. Символы '@' и '?' добавлены в те части кода, которые в дальнейшем будет необходимо заменить исходя из пожеланий пользователя. Для этого внесем следующий код в функцию main.

```
//выводим меню для пользователя  
printf("\n\t 1-сумма") ;  
printf("\n\t 2-максимум") ;  
printf("\n\t 3-минимум") ;  
printf("\nВведите тип искомого значения (см. список выше): ") ;  
int TIP ; scanf("%d",&TIP) ; if (TIP<1) TIP=1 ; if (TIP>3) TIP=3 ;  
//объявим новую строку opencl_source_dinamic для динамич.  
составления  
int len = strlen(opencl_source) ;  
char *opencl_source_dinamic ;  
opencl_source_dinamic = new char [len+50] ;  
//скомпонуем opencl_source_dinamic для решения нужной задачи  
int j=0 ;  
for (int i=0 ; i<len ; i++)  
{  
    if (opencl_source[i]!='@' && opencl_source[i]!='?')  
        { opencl_source_dinamic[j]=opencl_source[i] ; j++ ; }  
    if (opencl_source[i]=='@')  
        {  
            if (TIP==1) {opencl_source_dinamic[j]='0' ; j++ ; }  
            else { strcpy(&opencl_source_dinamic[j],"A[1]"); ; j+=4 ; }  
        }  
    if (opencl_source[i]=='?')  
        {
```

```

        if (TIP==1)
strcpy(&opencl_source_dinamic[j], "atomic_add");
        if (TIP==2)
strcpy(&opencl_source_dinamic[j], "atomic_max");
        if (TIP==3)
strcpy(&opencl_source_dinamic[j], "atomic_min");
        j+=10 ;
    }
} opencl_source_dinamic[j]='\0' ;
//заведём константный указатель на содержимое
opencl_source_dinamic
const char *opencl_source_const = opencl_source_dinamic;
printf("\n\n%s\n\n", opencl_source_const) ;
//далее создаем OpenCL программу на основе opencl_source_const

```

Порядок выполнения лабораторной работы

1) Разработайте OpenCL программу, решающую задачу из вашего варианта, используя события `cl_event` и `event_t`, а также динамическое составление кода функции-ядра исходя из входных данных. Учтите возможность разбиения OpenCL программы на **несколько рабочих групп**.

2) Протестируйте работу OpenCL программы и убедитесь, что она выполняется корректно.

Варианты заданий

Варианты	Входные данные	Вычислить/определить
1 15 29 43 57	[A, B], h, f(x)	площадь фигуры, заключенной между Oх и f(x).
2 16 30 44 58	[A, B], h, f(x), max или min	максимальное или минимальное значение f(x)
3 17 31 45 59	[A, B], h, f1(x), f2(x), e	точки в которых f1(x)=f2(x) с погрешностью e
4 18 32 46 60	[A, B], h, f(x), возрастание или убывание	является ли функция возрастающей или убывающей
5 19 33 47	[A, B], h, f(x)	протабулировать функцию
6 20 34 48	[A, B], h, f(x)	сколько раз функция меняет знак + на - и наоборот
7 21 35 49	[A, B], h, f(x), сумма или среднее	сумму значений функции или среднее арифметическое
8 22 36 50	[A, B], h, f(x), [C, D]	количество значений функции, попавших в интервал [C, D]

9 23 37 51	$[A, B], h, f(x)$	приблизенно длину графика функции
10 24 38 52	$[A, B], h, f(x), L$	локальные минимумы окрестности L
11 25 39 53	$[A, B], h, f(x), +$ или $-$	количество положительных или отрицательных значений функции
12 26 40 54	$[A, B], h, f(x), (X, Y)$	точку $(x, f(x))$ максимально близкую к (X, Y)
13 27 41 55	$[A, B], h, f_1(x), f_2(x)$	площадь фигуры, заключенной между $f_1(x)$ и $f_2(x)$
14 28 42 56	$[A, B], h, f_1(x), f_2(x), f_3(x), (X, Y)$	какая из трех функций проходит максимально далеко от (X, Y)

Лабораторная работа №6

Тема: Распараллеливание сложных алгоритмов.

Цели: Получить навык распараллеливания алгоритмически сложной задачи для выполнения на графическом процессоре.

Пример 1

Рассмотрим следующую задачу комбинаторного перебора. Имеется три массива чисел произвольной размерности A, B, C. Необходимо составить новый массив D, каждый элемент которого является суммой всех возможных троек $A[i]+B[j]+C[k]$.

Рассмотрим частный случай решения задачи.

$A = \{ 1, 2, 3 \}$, $B = \{ 4, 5 \}$, $C = \{ 6, 7, 8 \}$.

Тогда $D = \{ 1+4+6, 1+4+7, 1+4+8, 1+5+6, 1+5+7, 1+5+8, 2+4+6, 2+4+7, 2+4+8, 2+5+6, 2+5+7, 2+5+8, 3+4+6, 3+4+7, 3+4+8, 3+5+6, 3+5+7, 3+5+8 \}$

Размерность D составляет 18 элементов. Размерности исходных множеств A, B, C равны 3, 2, 3 элементам соответственно. $18 = 3 * 2 * 3$.

Реализуем последовательный алгоритм.

```
int A[3] = { 1, 2, 3 }; int NA = 3 ; // NA - число элементов в A
int B[2] = { 4, 5 }; int NB = 2 ; // NB - число элементов в B
int C[3] = { 6, 7, 8 }; int NC = 3 ; // NC - число элементов в C
int *D ; D = new int [NA*NB*NC] ;
int i, j, k, l=0 ; //i для перебора индексов в A, j в B, k в C, l в D
for (i=0 ; i<NA ; i++) //перебор всех элементов в A
for (j=0 ; j<NB ; j++) //перебор всех элементов в B
for (k=0 ; k<NC ; k++) //перебор всех элементов в C
{ D[l]= A[i]+B[j]+C[k] ; l++ ; } //помещение элемента в массив D
```

Разделим выполнение задачи между несколькими рабочими элементами. Для этого достаточно разделить между ними итерации только одного, например, первого цикла. Само разделение можно провести по принципу, отраженному на рис. 13.

Дополнительно необходимо определить разделение массива D между рабочими элементами так, чтобы они не обращались к одним и тем же индексам массива, тем самым переписывая чужие результаты. Вычисление индекса l можно производить не последовательным его увеличением на единицу, а через формулу, зависящую от i, j, k, это $l=i*NB*NC+j*NC+k$, здесь i умножается на NB*NC потому что между инкрементацией i на еди-

ницу производится NB итераций второго цикла и NC итераций третьего цикла.

Используя приведенные соображения составим код для обобщенного рабочего элемента (применяется только одно измерение).

```
int id=get_global_id(0), kol=get_global_size(0) ;  
for (int i=id ; i<NA ; i=i+kol)  
for (int j=0 ; j<NB ; j++)  
for (int k=0 ; k<NC ; k++)  
{ D[i*NB*NC+j*NC+k] = A[i]+B[j]+C[k] ; }
```

Пример 2

Рассмотрим приведенную выше задачу, при условии, что число исходных множеств и количество чисел в каждом множестве заранее не известно. В этом случае и в последовательном и в параллельном алгоритме применяется рекурсивная функция для перебора элементов.

Последовательный алгоритм.

```
#include "stdafx.h"  
#include "stdio.h"  
#include "locale.h"  
  
//в двумерном массиве A записаны множества  
int A[100][100], *C, *D ;  
//в массиве B записаны размерности этих множеств  
//массив C - искомое множество  
//массив D - вспомогательный массив для рекурсии,  
//куда вписываются индексы элементов для суммы  
int N ; //N-кол-во множеств  
int NN ; //NN-кол-во элементов в искомом множестве C  
  
int indC=0 ; //индекс для записи рекурсией в массив C  
void Rec(int indM, int *D) ; //заголовок рекурсивной функции  
//indM - индекс (номер) текущего обрабатываемого множества  
//D - хранит индексы элементов из множеств A  
//для их последующей суммы  
  
int _tmain(int argc, _TCHAR* argv[])  
{  
    setlocale(LC_ALL, "Russian") ;  
    printf("\nВведите кол-во множеств N=") ; scanf("%d",&N) ;
```

```

if (N<2) N=2 ; if (N>100) N=100 ;
int i,j ;
//ВВОД МНОЖЕСТВ
for (i=0 ; i<N ; i++) //цикл перехода по множествам
{
    printf("\nВведите кол-во элементов в множестве №%d =
",i) ;

    scanf("%d",&B[i]) ;
    if (B[i]<1) B[i]=1 ; if (B[i]>100) B[i]=1 ;
    //цикл перехода по элементам i-го множества
    for (j=0 ; j<B[i] ; j++)
    {printf("\nВведите элемент №%d из множества №%d =
",j,i) ;

        scanf("%d",&A[i][j]) ;
    }
}
//ВЫВОД МНОЖЕСТВ
printf("\n") ;
for (i=0 ; i<N ; i++)
{   printf("\nМножество   №%d   состоит   из   %d
элементов\n",i,B[i]) ;
    for (j=0 ; j<B[i] ; j++) printf("%d, ",A[i][j]) ;
}
//вычислим кол-во элементов NN в искомом множестве
NN=1 ;
for (i=0 ; i<N ; i++) NN=NN*B[i] ;
printf("\nКол-во элементов в искомом множестве NN=%d",NN) ;
C = new int [NN] ;
D = new int [N] ;
//запуск рекурсии от 0-го множества
Rec(0, D) ;
//вывод искомого множества C, найденного рекурсией
printf("\nC:") ;
for (i=0 ; i<NN ; i++) printf("%d, ",C[i]) ;
delete [NN] C ; delete [N] D ;
return 0;
}

```

```

void Rec(int indM, int *D)
{
    if (indM<N) //если еще не вышли за существующий номер
множества
        for (int i=0 ; i<B[indM] ; i++)
        { //ставим в D текущий индекс i из множества № indM
          D[indM]=i ; Rec(indM+1,D) ; //и сдвигаемся на следующее
множество
          //подавая в рекурсию формирующийся массив индексов D
        }
    else //вышли за существующий номер множества
    {
        //печатаем из каких индексов будет складываться сумма
        printf ("\n индексы ") ;
        for (int i=0 ; i<N ; i++) printf("%d, ",D[i]) ;
        //находим саму сумму
        C[indC]=0 ;
        for (int i=0 ; i<N ; i++) C[indC]+=A[i][D[i]] ;
        printf("результат сложения %d, ",C[indC]) ;
        indC++ ; //повышаем индекс элемента в массиве ответов
C
    }
}

```

Запустим программу и убедимся в ее работоспособности (рис. 17).

```

Введите кол-во множеств N=3
Введите кол-во элементов в множестве №0 = 2
Введите элемент №0 из множества №0 = 1
Введите элемент №1 из множества №0 = 2
Введите кол-во элементов в множестве №1 = 3
Введите элемент №0 из множества №1 = 10
Введите элемент №1 из множества №1 = 20
Введите элемент №2 из множества №1 = 30
Введите кол-во элементов в множестве №2 = 2
Введите элемент №0 из множества №2 = 100
Введите элемент №1 из множества №2 = 200

Множество №0 состоит из 2 элементов
1, 2,
Множество №1 состоит из 3 элементов
10, 20, 30,
Множество №2 состоит из 2 элементов
100, 200,
Кол-во элементов в искомом множестве NN=12
индексы 0, 0, 0, результат сложения 111,
индексы 0, 0, 1, результат сложения 211,
индексы 0, 1, 0, результат сложения 121,
индексы 0, 1, 1, результат сложения 221,
индексы 0, 2, 0, результат сложения 131,
индексы 0, 2, 1, результат сложения 231,
индексы 1, 0, 0, результат сложения 112,
индексы 1, 0, 1, результат сложения 212,
индексы 1, 1, 0, результат сложения 122,
индексы 1, 1, 1, результат сложения 222,
индексы 1, 2, 0, результат сложения 132,
индексы 1, 2, 1, результат сложения 232,
С:111, 211, 121, 221, 131, 231, 112, 212, 122, 222, 132, 232,

```

Рис. 17. Выполнение программы

Рассмотрим способ распараллеливания приведенного последовательного алгоритма с помощью рабочих элементов OpenCL. Применяется только одно измерение, всего запускается kol рабочих элементов. kol бессмысленно указывать больше чем количество элементов в первом (№0) множестве, т.к. рабочие элементы будут начинать рекурсивный процесс со своих индивидуальных элементов из первого множества. Из-за особенностей работы с многомерными массивами, если в последовательной программе использовалось A[100][100]. то в OpenCL массив A будет являться одномерным на [100*100] ячеек. Массив C не применяется, вместо него используется прямой вывод.

```

//заголовок рекурсивной функции
void Rec(int indM, int *D, __global int *A, __global int *B, int N) ;
//функция-ядро
__kernel void Kernel(__global int *A, __global int *B, int N)
{
    int i, id=get_global_id(0), kol=get_global_size(0) ;
    int D[100] ; //D - частный массив каждого рабочего элемента

```

```

        for (i=id ; i<B[0] ; i=i+kol) { D[0]=id ; Rec(1,&D,A,B,N); }
        //B D[0] ставим индекс "своего" элемента из первого
множества
        //далее запускаем заполнение последующих индексов в D -
Rec(1,...)
    }
    void Rec(int indM, int *D, __global int *A, __global int *B, int N)
    {
        if (indM<N) //если еще не вышли за существующий номер
множества
            for (int i=0 ; i<B[indM] ; i++)
            { //ставим в D текущий индекс i из множества № indM
              D[indM]=i ; Rec(indM+1,D,A,B,N) ;
            }
            //и сдвигаемся на следующее множество
            //подавая в рекурсию формирующийся массив индексов D
        }
        else //вышли за существующий номер множества
        {
            //находим сумму элементов по индексам из D
            int C=0 ;
            for (int i=0 ; i<N ; i++) C+=A+i*100+D[i] ;
            //в посл. алгоритме C+=A[i][D[i]] ;
            //A[i][j] <---> A + i*M + j, где M-число столбцов
            printf("\n %d ", C) ;
        }
    }
}

```

Порядок выполнения лабораторной работы

1) Разработайте OpenCL программу, решающую задачу из вашего варианта, используя события `cl_event` и `event_t`, а также динамическое составление кода функции-ядра исходя из входных данных. Учтите возможность разбиения OpenCL программы на **несколько рабочих групп**.

2) Протестируйте работу OpenCL программы и убедитесь, что она выполняется корректно.

Варианты заданий

1. Из заданного множества точек на плоскости произвольной размерности, найти расстояния между всеми возможными парами точек, образующими вектора ненулевой длины.

2. Заданы n пищевых продуктов, содержащих m различных питательных веществ. Обозначим через a_{ij} содержание (долю) j -го питательного

вещества в i -ом продукте, через b_j — суточную потребность организма в j -ом питательном веществе, через c_i — стоимость единицы i -го продукта. Составить суточный рацион питания минимальной стоимости, удовлетворяющий потребность во всех питательных веществах.

3. Задан двумерный численный массив размерности $N \times K$. Найти индексы (i_1, j_1, i_2, j_2) , между которыми заключена наиболее длинная цепочка одинаковых элементов, расположенных по диагонали. (Например в

0 1 2 3
5 1 1 3
4 5 2 1

массиве **1 0 2 3** ответом является (от 0-й строки, 1 столбца, до 2-й строки, 3 столбца)).

4. Заданы n точек в двумерном пространстве. Найти точку, лежащую в прямоугольнике с верхним левым углом в точке (a, b) , длиной s и высотой d , сумма расстояний от которой до каждой из заданных n точек будет минимальной.

5. Найти все N -значные числа (N указывает пользователь), чья обратная запись совпадает с их значением. (Например числа 111 и 515 подходят под искомое условие, а 428 нет.)

6. Задан двумерный численный массив размерности $N \times N$ и число K . Найти такую перестановку строк массива, чтобы на главной диагонали расположились элементы, в сумме дающие K .

7. В файле задано произвольное количество интервалов $[A, B]$, среди них могут присутствовать интервалы вида $[-\infty, B]$, $[A, +\infty]$, $[-\infty, +\infty]$. Найти интервал минимальной длины, покрывающий все интервалы, заданные в файле. Найти интервал-пересечение всех интервалов из файла или убедиться, что такого пересечения не существует.

8. Задан набор из n городов. Известны стоимости строительства транспортных маршрутов между каждыми из n городов. Составить такой маршрут, чтобы общая его стоимость была минимальна, и существовал путь для каждой пары городов из набора (не обязательно прямой).

9. Задан массив чисел, найти все возможные особые тройки чисел (A, B, C) из элементов заданного массива. Тройка чисел считается особой, если в ней нет одинаковых чисел и наибольший общий делитель всех трех чисел равен единице.

10. Задана матрица чисел A размерности $m \times n$ и точность ϵ . Итерация алгоритма состоит в вычислении всех не граничных элементов новой матрицы по следующей формуле $A'_{i,j} = (A_{i+1,j} + A_{i-1,j} + A_{i,j+1} + A_{i,j-1})/4$.

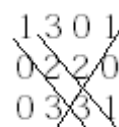
Итерация алгоритма будет последней, если для каждой пары не граничных элементов из вычисленной матрицы X и Y , выполняется $|X-Y| < \epsilon$.

11. Найти определитель матрицы A . Входные данные: целое положительное число n , произвольная матрица A размерности $n \times n$.

12. Задан одномерный массив чисел A размерности N . Найти из элементов массива A все возможные четверки чисел (r, a, b, c) где $r = A * a + B * b + C * c$. Коэффициенты A, B, C вводятся пользователем, искомые r, a, b, c являются элементами массива A , при этом отдельные или все значения из r, a, b, c могут быть равны одному и тому же элементу массива A .

13. Задача о женихах и невестах. Имеется n женихов и m невест, каждый жених имеет мнение о каждой из невест, это мнение выражено как балл от 0 до 10. Аналогично каждая невеста имеет мнение о каждом женихе. Требуется составить k пар (или проверить, что составить k пар невозможно), таким образом, чтобы не существовало никаких других потенциальных пар, для которых сумма баллов жениха и невесты была бы большей, чем у образованных пар. При составлении пары следует учитывать, что пара не может быть создана, если бал жениха относительно невесты меньше 5 или наоборот.

14. Задан двумерный численный массив A размерности $N \times M$ и одномерный массив-шаблон B размерности K , найти число вхождений массива-шаблона B в прямой и обратной записи в диагональных и параллельных им

последовательностях элементов массива A . (Например для A  и $B = \{1, 2, 3\}$ число вхождений B в $A = 3$.)

15. Найти алгебраическое дополнение для каждого элемента матрицы. Входные данные: целое положительное число n , произвольная матрица A размерности $n \times n$.

16. Задан набор из N генов, каждый ген обозначается с помощью индивидуального символа. Определить по самостоятельно выбранному принципу все возможные комбинации взаимодействия пар генов (например для трех генов: $A+A=A$, $A+B=C$, $A+C=B$, $B+B=C$, $B+C=A$, $C+C=B$). Задана цепочка из K генов родителя 1 и цепочка такой же размерности генов родителя 2. Определить вид цепочки генов потомка родителя 1 и родителя 2. (Необходимо распараллелить обе части задачи - формирование правил и вычисление генов потомка)

17. Найти все особые N-значные числа. N и K (см. ниже) указывает пользователь. Число является особым если делиться нацело на обратную запись хотя бы одного своего делителя большего K. (Например число 504 имеет делитель 21 (пусть $K=20$, $21>K$), обратная запись 21 это 12, число 504 делится нацело и на 12, значит 504 является особым числом.)

18. В двумерном массиве чисел A размерности $N \times N$ вычислить выражение $A[0][0] - A[?][?] + A[?][?] - A[?][?] + \dots \pm A[?][?]$. Коэффициенты в выражении определить обходом всех элементов массива в следующем по-

рядке.

1	2	3	4
12	13	14	5
11	16	15	6
10	9	8	7

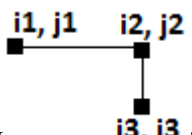
19. В файле задано множество из L двумерных массивов чисел размерности $N \times K$ (при этом размерности разных массивов могут не совпадать). Вывести на экран и в новый файл те массивы, чьи суммы элементов превышают указанное пользователем число S.

20. Задан двумерный численный массив размерности $N \times K$. Найти группу индексов ($i1, j1, i2, j2, i3, j3$) между которыми заключено наибольшее количество одинаковых элементов, расположенных в форме буквы Г.

(Например в массиве

0	2	0	2	0
0	1	1	1	0
2	0	2	1	1
0	2	0	1	1

искомые индексы



это (1, 1, 1, 3, 3, 3).)

21. Найти все совершенные N-значные числа. N указывает пользователь. Совершенное число равно сумме всех своих делителей (не включая делитель равный ему самому).

22. Из одномерного массива чисел произвольной размерности найти все возможные пары чисел (A, B), где $A \neq B$ и A и B имеют наибольший общий делитель (НОД) > 2 .

23. Вычислить прямое произведение множеств A_1, A_2, A_3, A_4 . Входные данные: множества чисел A_1, A_2, A_3, A_4 , мощности множеств могут быть не равны между собой и мощность каждого множества ≥ 1 . (Прямое произведение множеств это не нахождение произведения всех элементов, это математический термин, предполагающий построение нового множества по определенным правилам.)

24. Задан двумерный численный массив A размерности $N \times M$ и одномерный массив-шаблон B размерности K, найти число вхождений массива-

1	2	3	4
2	0	1	0
3	5	2	1
4	4	3	4

шаблона В в строках и столбцах массива А. (Например для А и В = {1, 2, 3} число вхождений В в А = 3.)

25. Заданы два численных массива А и В, размерности А и В могут не совпадать. Найти все пары (A[i], B[j]), в которых либо A[i] является кубом B[j], либо B[j] является кубом A[i].

26. Задан двумерный целочисленный массив размерности NхN. Найти такую перестановку строк массива (или убедиться что искомой перестановки не существует), чтобы сумма элементов, расположенных на главной диагонали совпала с суммой элементов, расположенных на побочной диагонали.

27. Вычислить прямое произведение множеств $A_1, A_2, A_3 \dots A_n$. Входные данные: целое положительное число n, множества чисел $A_1, A_2, A_3 \dots A_n$, мощности множеств равны между собой и мощность каждого множества ≥ 1 . (Прямое произведение множеств это не нахождение произведения всех элементов, это математический термин, предполагающий построение нового множества по определенным правилам.)

28. Задан двумерный массив целых чисел размерности NхK. Определить конечный стационарный вид массива, либо его вид на k-ой итерации ($k \geq 30.000$ и задается пользователем), если массив так и не пришел в стационарное состояние. Итерация состоит в том, что каждый элемент массива больший 10, вычитает от своего значения по единице и добавляет

+1	+1	+1
+1	-8	+1
+1	+1	+1

их в непосредственно окружающие его элементы массива

-3	+1
+1	+1

(учесть ситуацию граничных элементов), а элемент меньший -10, отнимает по единице от непосредственно окружающих его элементов, больших нуля и прибавляет к своему значению (также учесть ситуацию элемента на границе).

29. Задано множество точек на плоскости, для всех возможных пар не одинаковых точек определить коэффициенты уравнения прямой $y=kx+b$, проходящей через эту пару точек.

30. Задан двумерный численный массив размерности NхK. Найти индексы строки и столбца (i, j), являющиеся центром наибольшего количе-

ства одинаковых элементов, расположенных в форме буквы X. (Например в

0 5 0 5
1 5 5 1

массиве 0 5 1 5 искомые индексы это (1-я строка, 2-й столбец) при нумерации с нуля.)

31. Задано произвольно количество не совпадающих между собой символов. Найти все возможные слова, состоящие из K символов, не содержащие повторяющиеся символы, которые можно составить, пользуясь заданными символами. Число K задается пользователем.

32. Представить выражение $(ax + by)^n$ в виде $C_1a^n x^n + C_2b^n y^n + C_3a^{n-1}x^{n-1}by + C_4b^{n-1}y^{n-1}ax + C_5a^{n-2}x^{n-2}b^2y^2 + C_6b^{n-2}y^{n-2}a^2x^2 + \dots$. Входные данные: числа a и b , целое положительное число n .

33. Задан двумерный целочисленный массив размерности $N \times K$. Найти все пары индексов столбцов (i, j) , $i \neq j$, которые содержат одинаковое ненулевое количество элементов равных двойке в произвольной целой степени.

34. Из указанного пользователем интервала $[A, B]$, где A, B могут принимать и отрицательные значения найти все возможные тройки не одинаковых целых чисел, обладающие наибольшим общим делителем равным K . Целое число K указывается пользователем.

35. Найти все автоаморфные N -значные числа. N указывает пользователь. Автоаморфное число совпадает с последними разрядами своего квадрата. (Например 25, его квадрат 625, содержит само 25 в последних разрядах.)

36. Задан одномерный массив чисел, найти максимальное количество элементов из массива, образующих произвольную арифметическую последовательность вида $X_i = L + X_{i-1}$. Вывести найденные элементы в упорядоченном виде.

37. Задано множество точек на плоскости, вычислить круг минимального радиуса, покрывающий все заданные точки. Искомый круг обладает параметрами - координаты центра и радиус.

38. Определить, является ли множество C объединением множеств A и B ($A \cup B$), пересечением множеств ($A \cap B$), разностью множеств A и B ($A \setminus B$), разностью множеств B и A ($B \setminus A$). Входные данные: множества целых положительных чисел A, B, C .

39. Задано множество из четного числа игроков, каждый игрок обладает известным уровнем мастерства. При образовании пары игроков, каждый игрок получает удовлетворение от игры равное 2 если уровень

мастерства партнера совпадает с его мастерством в диапазоне ± 1 , либо равное 1 если уровень мастерства партнера выходит из диапазона ± 1 , но не превышает диапазона ± 3 , при выходе уровня мастерства игрока-партнера за диапазон ± 6 , удовлетворение от игры принимает отрицательное значение -1. Найти распределение пар игроков (при этом каждый игрок должен участвовать в игре), дающее в сумме максимальное удовлетворение от игры.

40. Задан двумерный численный массив размерности $N \times N$. Определить возможно ли, используя только одну перестановку двух столбцов между собою и только одну перестановку двух строк между собою, привести массив в состояние, чтобы на главной диагонали и на побочной диагонали элементы образовывали возрастающие последовательности.

41. Найти все возможные тройки компланарных векторов. Входные данные: множество векторов (x, y, z) , где x, y, z – числа.

42. Задан двумерный численный массив A размерности $N \times M$ и двумерный массив-шаблон B размерности $K \times Z$, найти число вхождений

1 2 1 1
3 4 1 2 1 2

массива-шаблона B в массиве A . (Например для $A = \begin{smallmatrix} 3 & 3 & 3 & 4 \end{smallmatrix}$ и $B = \begin{smallmatrix} 3 & 4 \end{smallmatrix}$ число вхождений B в $A = 2$.)

43. Из произвольного числа заданных окружностей, описанных через координаты центра и радиус, выделить все возможные пересекающиеся пары окружностей.

44. Определить индексы i, j ($i \neq j$), для которых выражение $A[i] - A[i+1] + A[i+2] - A[i+3] + \dots \pm A[j]$ имеет максимальное значение. Входные данные: массив чисел A произвольной длины.

45. В файле задано множество из L двумерных массивов чисел одинаковой размерности $N \times K$. На основе этих данных составить соответствующий трехмерный массив, в котором вычислить минимальный и максимальный элементы а также их индексы в трехмерном массиве.

46. Из заданного множества точек на плоскости произвольной размерности, выделить множество всех возможных троек точек $(x_1, y_1), (x_2, y_2), (x_3, y_3)$, которые образуют произвольный треугольник, вычислить площади всех таких треугольников.

47. Определить индексы i, j , для которых существует наиболее длинная последовательность $A[i] < A[i+1] < A[i+2] < A[i+3] < \dots < A[j]$. Входные данные: массив чисел A , произвольной размерности.

48. Найти все особые N-значные числа. N указывает пользователь. Особое число делится нацело на количество цифр $\neq 1$ в своем наибольшем делителе не равном самому этому числу. (Например для числа 32 наибольший делитель не совпадающий с ним самим это 16, количество цифр в 16 равно 2, 32 делится на 2 нацело, значит 32 является особым.)

49. Определить множество пар индексов (i, j) для которых $A[i] + B[j]$ являются простыми числами. Входные данные: массивы целых чисел A и B, их размерности.

50. Определить множество пар индексов (i, j) для которых $A[i]$ и $B[j]$ не имеют общих делителей (единицу в роли делителя не рассматривать). Входные данные: массивы целых чисел A и B, их размерности.

51. Вывести список всех целых чисел, содержащих от 4 до 9 значащих цифр, которые после умножения на n, будут содержать все те же самые цифры в произвольной последовательности и в произвольном количестве. Входные данные: целое положительное число n, больше единицы и меньше десяти.

52. Задано множество из N строк $S[N]$, сформировать множество строк $S2[N]$, записывая в элемент $S2[i]$ строку $S[i]$, с удалением из нее тех ее слов, которые встречаются хотя бы однажды в других строках $S[j]$ ($i \neq j$).

53. Заданы матрицы чисел A и B размерности $m \times n$ и точность ϵ . Итерация алгоритма состоит в переопределении элементов матрицы A по следующей формуле $A_{i,j} = (A_{i,j} + A_{i+1,j} + B_{i,j})/3$. Если для $A_{i,j}$ элемента не существует элемента $A_{i+1,j}$, то применяется формула $A_{i,j} = (A_{i,j} + B_{i,j})/2$. Итерация алгоритма будет последней, если для всех i, j $|A_{i,j} - B_{i,j}| < \epsilon$.

54. Вычислить произведение матриц $A1 * A2 * \dots * AN$. Входные данные: натуральные N, K, массивы чисел $A1, A2, \dots, AN$ размерности $K \times K$.

55. В двумерном массиве чисел размерности $N \times K$ найти количество контуров прямоугольников, состоящих из одинаковых значений. (Например

0	0	0	2	1	1	1	1	5	5	5	
0	9	0	2	1	2	3	2	1	5	6	5
0	7	0	4	1	3	3	5	1	5	7	5
0	0	0	4	1	4	4	3	1	5	5	5
2	0	3	2	1	4	4	5	1	7	8	0

в массиве

3	1	0	0	1	1	1	1	1	1	5	9
---	---	---	---	---	---	---	---	---	---	---	---

 количество таких контуров равно четырем, это контуры составленные из: 0, 1, 4, 5.)

56. Задан двумерный численный массив размерности $N \times N$, число K. Найти все содержащиеся в заданном массиве квадратные подмассивы,

1	2	0	1
2	1	0	1
0	0	2	2
0	0	1	1

сумма элементов которых равна K . (Например для массива и $K=6$ можно найти три подмассива, два из которых размерности 2×2 и один размерности 3×3 .)

57. Найти все N значные числа, содежащие K раз цифру L , и не держащие ни одной из цифр $M_1, M_2, \dots M_x$. Значения $N, K, L, x, M_1, M_2, \dots M_x$ указываются пользователем.

58. Задан двумерный целочисленный массив размерности $N \times N$. Найти такую перестановку столбцов массива (или убедиться что искомой перестановки не существует), чтобы по главной диагонали массива разместились элементы, кратные K . Целое число K задается пользователем.

59. В множестве целых чисел произвольной размерности вычислить все общие делители всех заданных чисел.

60. Из множества чисел произвольной размерности ≥ 5 вывести в упорядоченном виде все выборки не менее чем из пяти элементов ($X_1, X_2, X_3, X_4, X_5, \dots$), образующие произвольную геометрическую последовательность $X_i = L * X_{i-1}$.